6주차 강의

OPC UA 개요

OPC Unified Architecture의 개념과 구조

김한진
gks359@cbnu.ac.kr
충북대학교 산업인공지능연구센터

INDUSTRIAL AI RESEARCH CENTER

CONTENTS

I OPC UA 개념

II OPC Foundation

III 정보 모델링

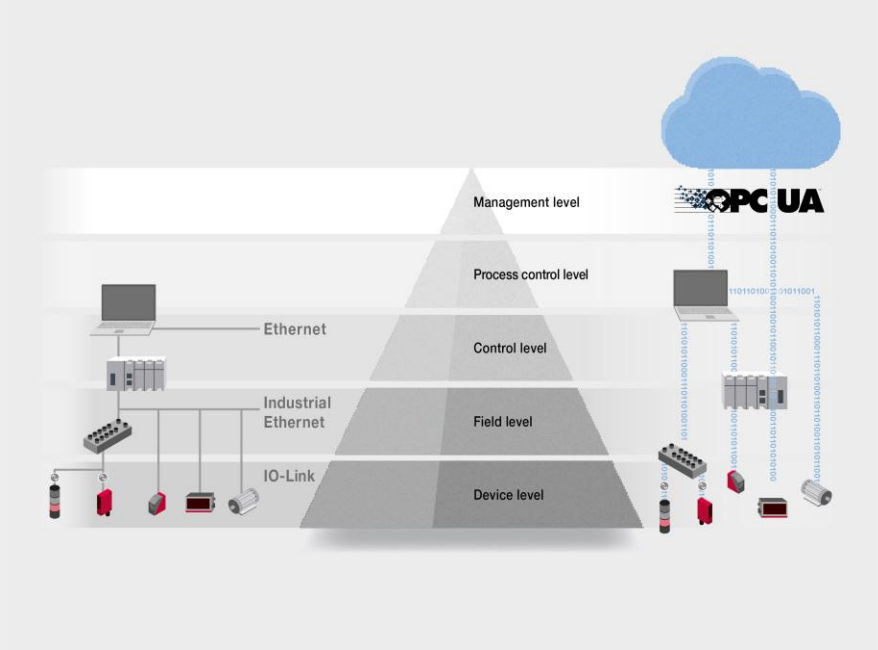
IV 시스템 및 서비스

■ 정의 및 범위

➢ OPC UA는 산업 데이터를 의미·보안과 함께 교환하는 플랫폼 독립 표준

OPC Unified Architecture

OPC UA는 산업 자동화 및 타 산업 분야에서 안전하고 신뢰성 있는 데이터 교환을 위한 상호운용성 표준
OPC UA는 플랫폼 독립적이면서, 다수의 벤더들이 제공하는 이기종 장치 간 막힘 없는 정보 교환을 보장



■ 정의 및 범위

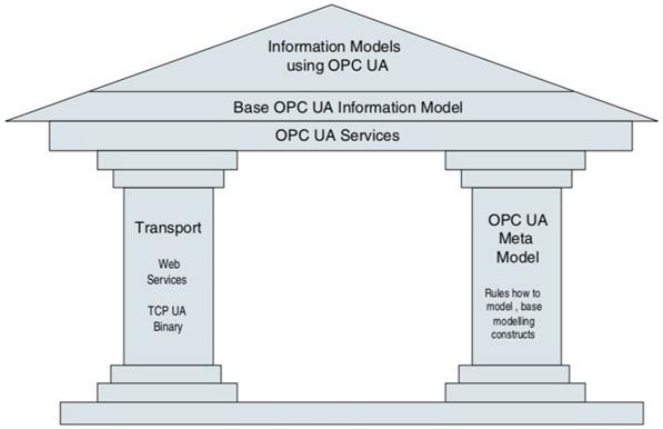
➢ OPC UA는 산업 데이터를 의미·보안과 함께 교환하는 플랫폼 독립 표준

OPC Unified Architecture

OPC UA는 산업 자동화 및 타 산업 분야에서 안전하고 신뢰성 있는 데이터 교환을 위한 상호운용성 표준
OPC UA는 플랫폼 독립적이면서, 다수의 벤더들이 제공하는 이기종 장치 간 막힘 없는 정보 교환을 보장

Unified의 의미: 따로 있던 Classic 사양을 하나로

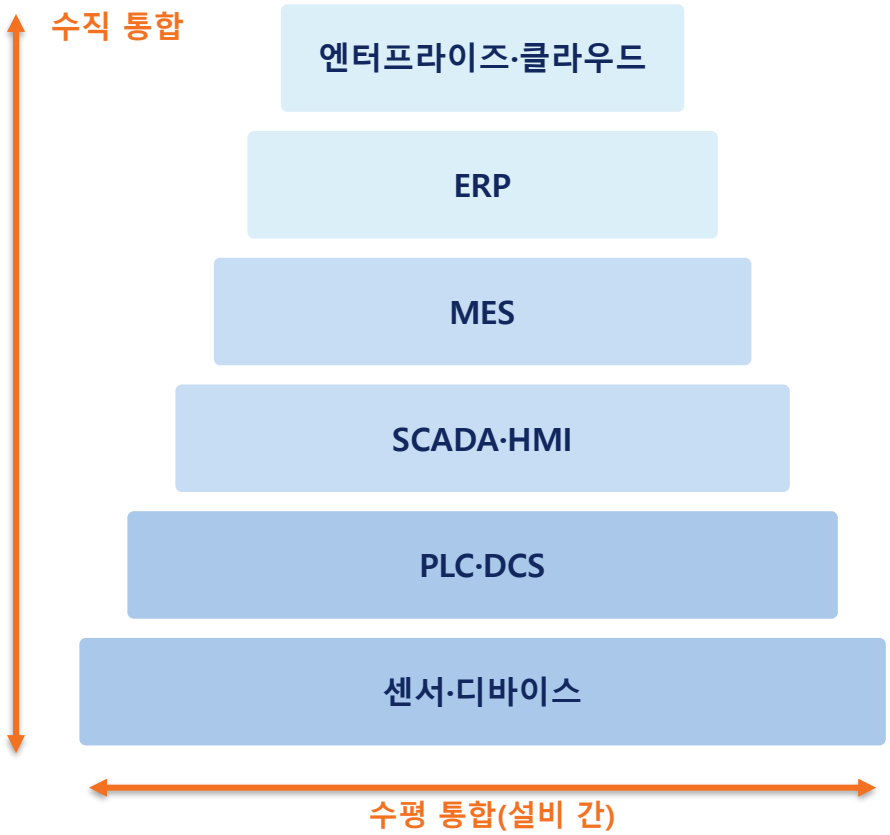
DA	HDA	A&E	→	하나의 통합 주소공간
실시간 데이터	이력 데이터	알람·이벤트		



OPC UA 개념

■ 정의 및 범위

➢ 센서·디바이스부터 클라우드까지 같은 방식으로 연결



OPC UA가 정의하는 것

- 데이터 교환 서비스와 메시지 형식
- 정보 모델링 규칙과 표준 정보 모델
- 보안 모델(인증·서명·암호화·감사)
- 디스커버리·인증서 관리 등 인프라

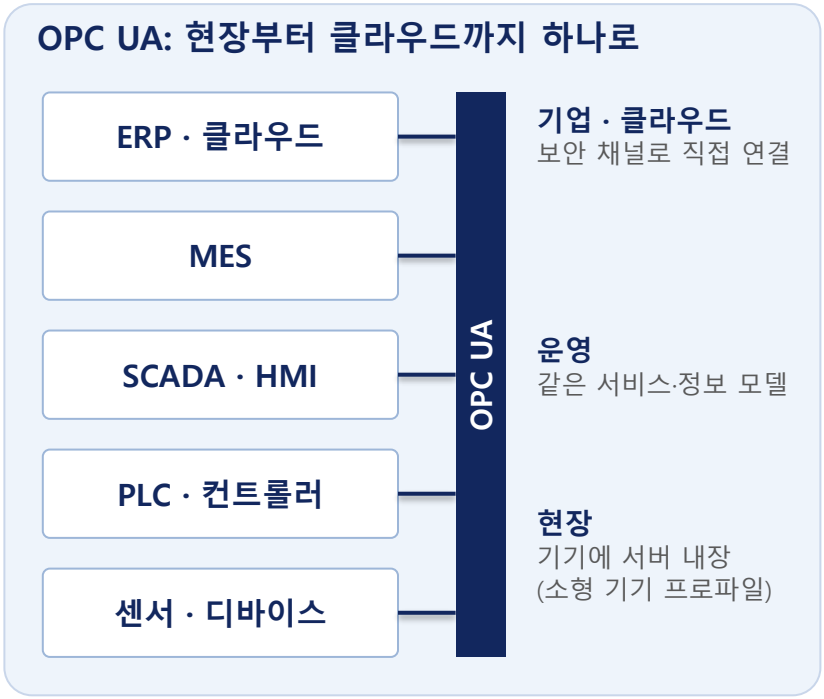
정의하지 않는 것

- 제어 로직과 알고리즘
- HMI 화면 등 애플리케이션 기능
- 특정 하드웨어·운영체제

■ OPC UA 기반 수직적 통합

➢ 계층마다 다른 프로토콜 대신, 현장에서 클라우드까지 OPC UA 하나로 구성





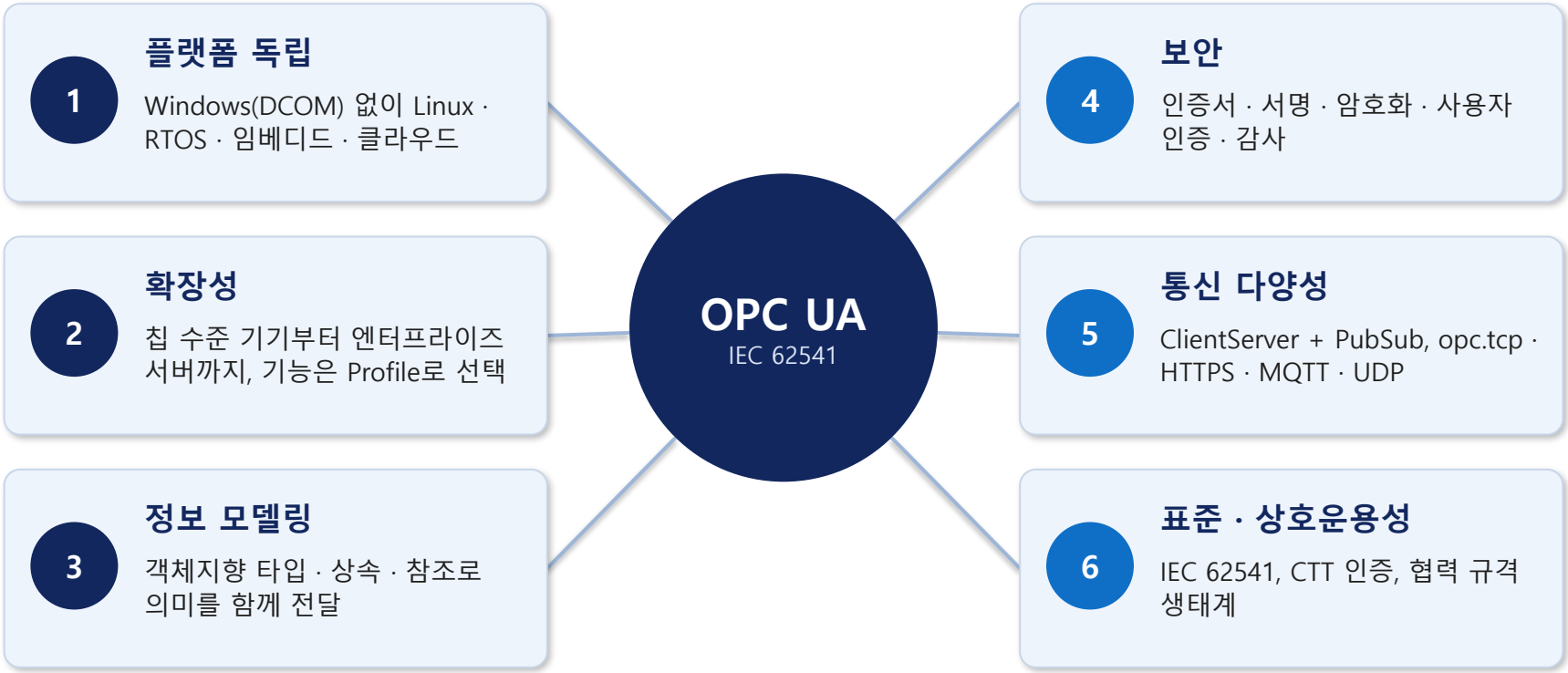
■ 특징

➢ OPC Classic의 한계를 플랫폼 독립·통합 모델·보안으로 풀어냄

구분	OPC Classic	OPC UA
기반 기술	Windows COM/DCOM	플랫폼 독립(TCP·HTTPS·WebSockets)
데이터 모델	DA·HDA·A&E 사양별로 분리	하나의 통합 주소공간
데이터 의미	태그 이름과 값 위주	타입·구조·참조로 의미 표현
보안	DCOM·Windows 설정에 의존	인증서 기반 인증·서명·암호화
원격 접속	DCOM 포트·방화벽 문제	단일 포트(기본 4840)로 접속
적용 범위	PC 기반 HMI·SCADA	임베디드 기기부터 클라우드까지

■ OPC UA 특징

➢ 플랫폼 독립·확장성·정보 모델·보안·통신 다양성·표준화가 핵심



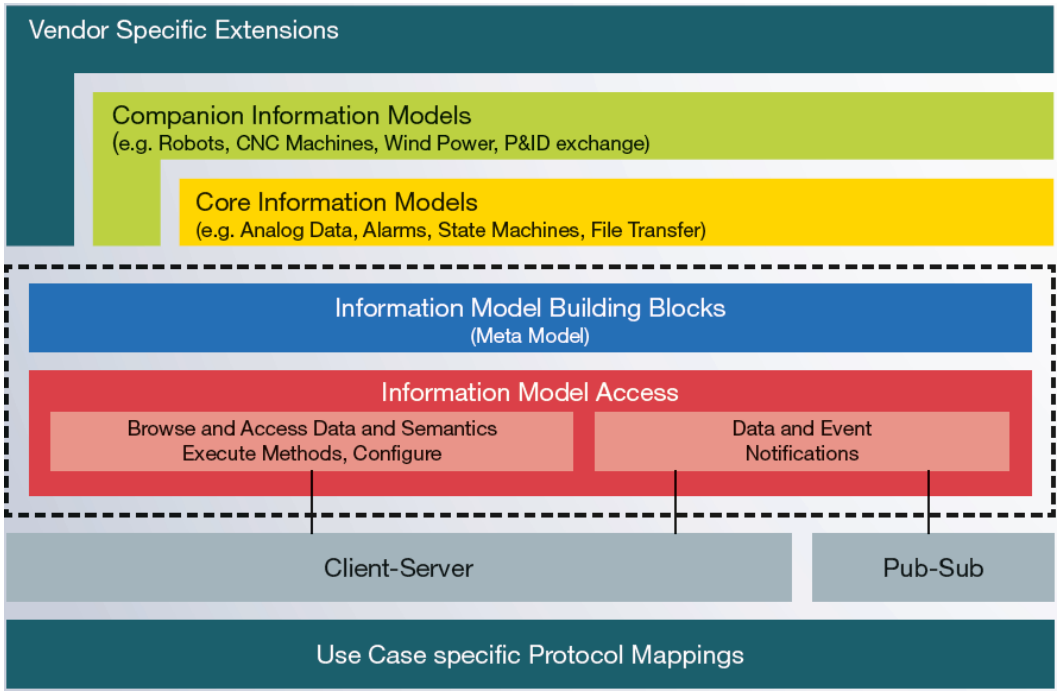
■ 특징

➢ Modbus·MQTT와 같은 미들웨어와 비교하면 '의미를 함께 보낸다'가 핵심 차이

구분	Modbus TCP	MQTT	OPC UA
데이터 의미	레지스터 주소만	페이로드 형식 미정의	정보 모델로 타입·단위·구조 전달
통신 모델	요청-응답(폴링)	Pub/Sub(브로커)	ClientServer + PubSub
탐색	불가	토픽 구독으로 일부	Browse로 주소공간 탐색
보안	기본 사양에는 없음	TLS, 브로커 계정	인증서 인증, 서명·암호화, 사용자 인증
주 용도	필드 기기 단순 연결	경량 원격 수집	설비·시스템 간 의미 기반 연계

■ OPC UA 개념 구조

➢ 전송과 메타모델을 기반으로, 서비스와 정보 모델을 층으로 구성



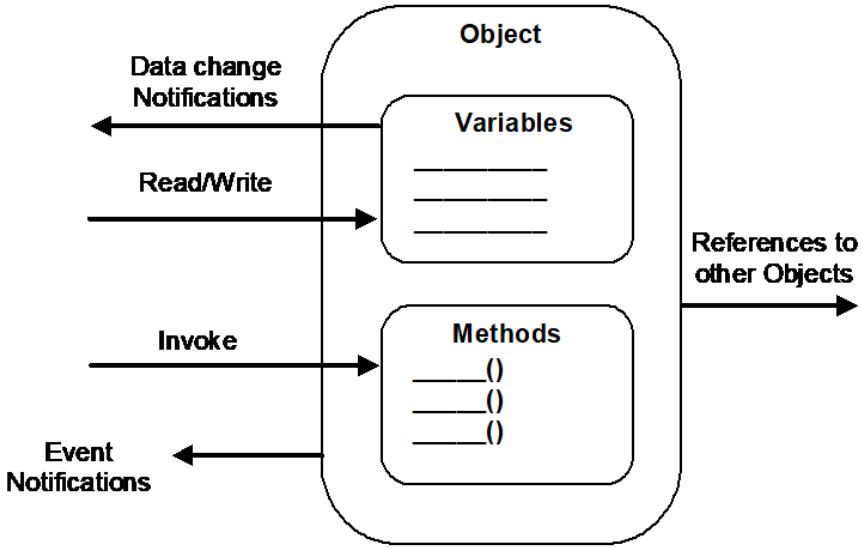
확장 영역
산업 · 벤더가 정보 모델을 확장

공통 영역
어떤 모델이든 같은 서비스로 접근

기반
어떻게 보내고 어떻게 표현할지 정의

■ OPC UA 객체 모델

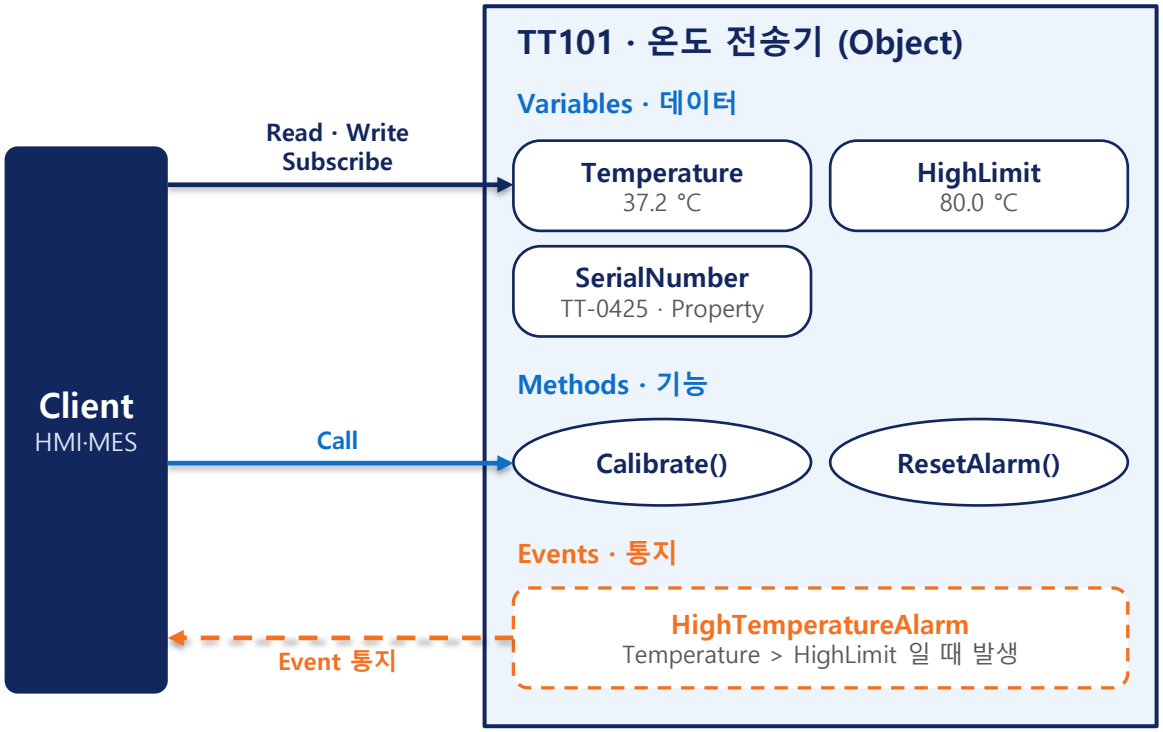
➢ 객체는 변수(데이터)·메서드(기능)·이벤트(통지)로 표현



원문: OPC 10000-3 Figure 2 – OPC UA Object Model · 근거: OPC 10000-1 §4.4.3 Integrated object model

■ OPC UA 객체 모델

➢ 객체는 변수(데이터)·메서드(기능)·이벤트(통지)로 표현



Object = Variable + Method + Event
다른 Object와의 관계도 함께 표현한다

주소공간에서는 모두 Node
NodeClass · 속성 · 참조로 기술한다 (피장)

하나의 모델로 통합
값 · 알람 · 이벤트 · 이력을 한 서버에서 (Classic DA · A&E · HDA 통합)

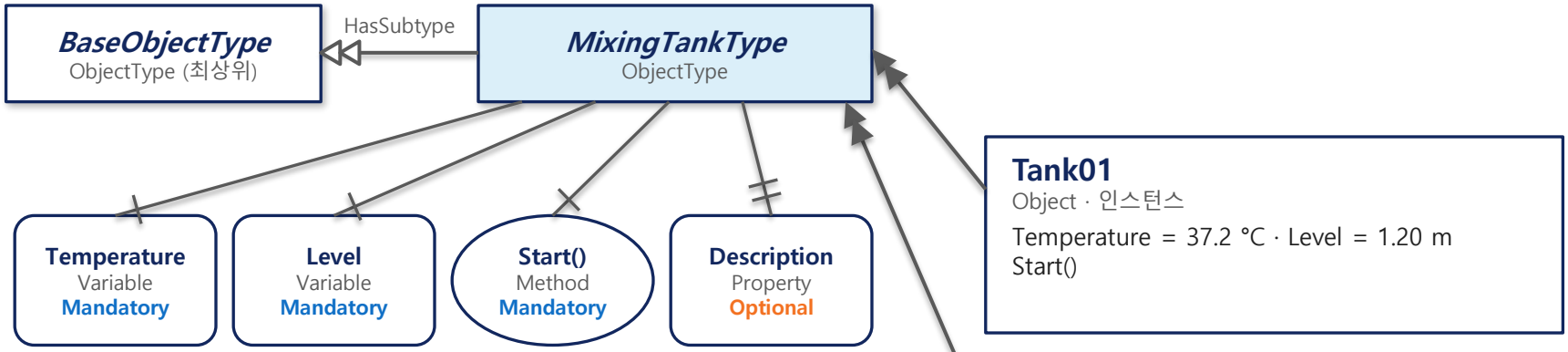
원문: OPC 10000-3 Figure 2 – OPC UA Object Model · 근거: OPC 10000-1 §4.4.3 Integrated object model

https://www.youtube.com/watch?v=vrhIxBWSJ04

INDUSTRIAL AI RESEARCH CENTER

■ OPC UA 정보 모델링

➢ 객체지향(Object-Oriented) 개념에 기반해 의미론적(Semantic)으로 디지털 주소 공간(Address Space)에 정의하는 과정



타입 정의 = 구조의 틀 (클래스)

- Mandatory는 모든 인스턴스가, Optional은 필요할 때만 갖는다
- 인스턴스는 HasTypeDefinition으로 자기 타입을 가리킨다
- HasSubtype으로 상위 타입을 상속하고 요소를 더한다

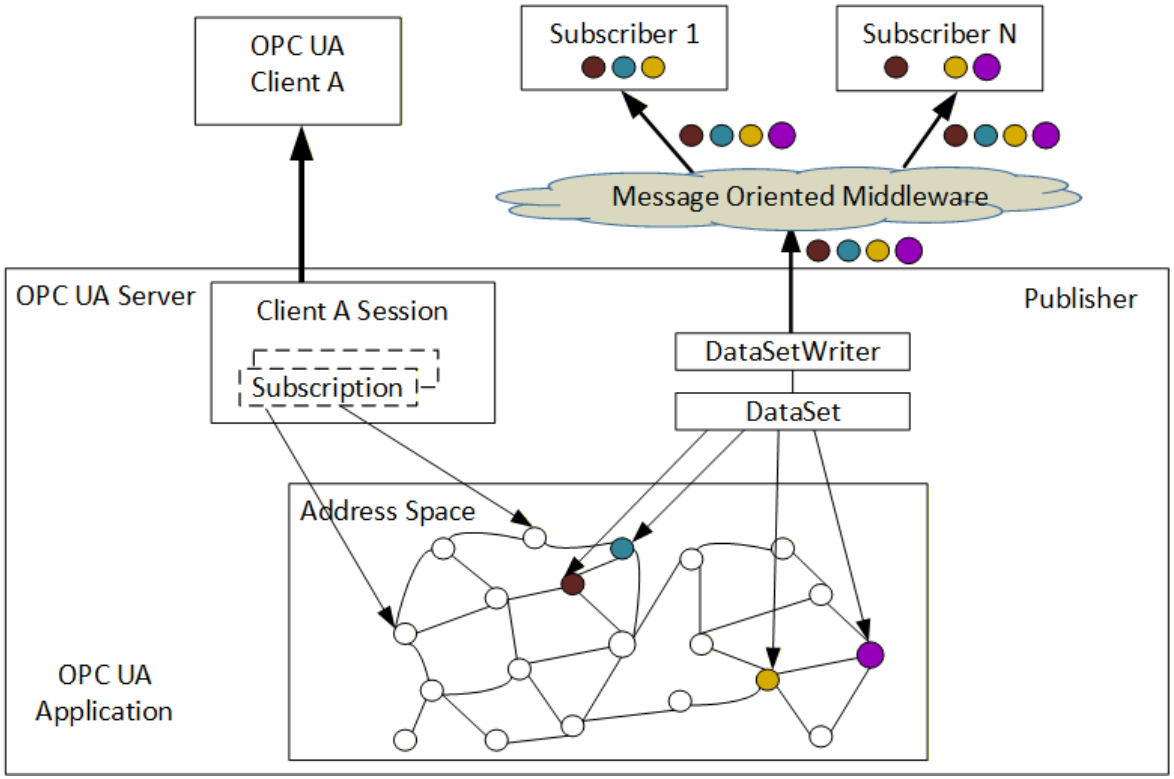
인스턴스 = 실제 설비 (객체)



원문: OPC 10000-3 Figure 7 – Object and its Components defined by an ObjectType · §4.6.1

■ 시스템

➢ 정보 모델에 정의된 데이터를 안전하고 신뢰성 있게 수집·제어·교환하기 위한 아키텍처



OPC UA pub-sub – client-server 통합 아키텍처

INDUSTRIAL AI RESEARCH CENTER

■ 규격과 협력 규격

➢ 협력 규격(Companion Specification)은 산업별 정보 모델을 표준화

벤더·사용자 모델
예: MixingTankType

산업별 협력 규격
Robotics · PackML · EUROMAP 77 · PA-DIM 등

공통 협력 규격
DI(디바이스) · Machinery(기계 공통)

OPC UA 코어 규격
OPC 10000 시리즈 (IEC 62541)

공동 작업반(JWG)에서 함께 개발

- **VDMA** · 기계(Machinery, Robotics 등)
- **PLCopen** · 제어기 프로그래밍
- **EUROMAP** · 플라스틱·고무 기계
- **FieldComm Group** · 공정 계측(PA-DIM)

배포물

- 사양 문서 + **NodeSet2 XML**
- UA-Nodeset 저장소, 온라인 레퍼런스

같은 종류의 설비는 같은 모델 → **벤더가 달라도 같은 클라이언트 코드로 접근**

https://www.youtube.com/watch?v=odDQ83bzoWE&t=6s

INDUSTRIAL AI RESEARCH CENTER

■ 규격과 협력 규격

➢ OPC 10000 시리즈(OPC UA 표준 번호)는 기능별 파트로 나뉘고 IEC 62541로 표준화됨

기본·핵심	접근 유형·모델	인프라·통신	공통 기능
Part 1 개요와 개념	**Part 8** 데이터 접근	**Part 12** 디스커버리·글로벌	**Part 15** 안전
Part 2 보안 모델	**Part 9** 알람·컨디션	**Part 14** PubSub	**Part 19** 사전 참조
Part 3 주소공간 모델	**Part 10** 프로그램	**Part 17** 별칭 이름	**Part 20** 파일 전송
Part 4 서비스	**Part 11** 이력 접근	**Part 18** 역할 기반 보안	**Part 23** 공통 참조 타입
Part 5 정보 모델	**Part 13** 집계	**Part 21** 디바이스 온보딩	**Part 24** 스케줄러
Part 6 매핑	**Part 16** 상태 기계	**Part 22** 기본 네트워크 모델	**Part 25** 객체 직렬화
Part 7 프로파일			**Part 26** LogObject

Part 100 디바이스(DI) · 협력 규격의 공통 기반

Part 80~84 OPC UA FX · 필드 레벨 통신

INDUSTRIAL AI RESEARCH CENTER

CONTENTS

I OPC UA 개념

II OPC Foundation

III 정보 모델링

IV 시스템 및 서비스

■ 목적

➢ 사용자·벤더·단체가 함께 산업 상호운용 표준을 만든다

미션(요지)
사용자·벤더·컨소시엄이 협력해, 산업 자동화에서 **멀티벤더·멀티플랫폼의 안전하고 신뢰할 수 있는 상호운용**을 위한 데이터 전송 표준을 만드는 글로벌 조직

1 규격 개발·유지

- OPC UA 코어 규격과 협력 규격을 만들고 개정한다
- 예: 1.05.0x 유지보수 릴리스

2 적합성 인증

- CTT(적합성 시험 도구)와 독립 시험소로 제품을 인증한다
- Profile 단위로 시험

3 표준화 협력

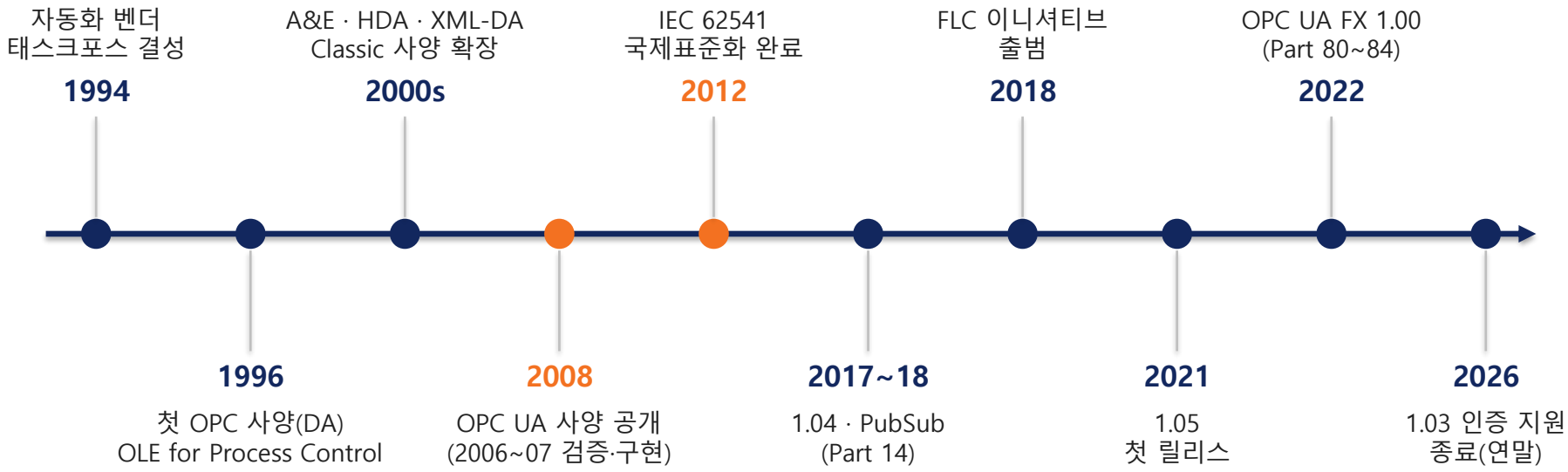
- IEC 국제표준화(IEC 62541)
- 산업 단체와 공동 작업반 운영

회원 1,000곳 이상 (OPC Foundation 소개 자료 기준)

출처: opcfoundation.org Mission Statement

■ 역사

➢ OPC Classic(1996)에서 OPC UA(2008), OPC UA FX(2022)로 진화했다



OPC = OLE for Process Control(초기) → **Open Platform Communications**(현재)

출처: OPC Foundation 소개 자료·보도자료, Wikipedia(OPC Foundation)

■ 조직

➢ 회원이 참여하는 이사회·자문위원회·작업반·인증 프로그램으로 운영된다



출처: opcfoundation.org Organization

■ 조직 · 규격 개발 과정

➢ 규격은 작업반 개발 → 검토·릴리스 → 공개 → 인증 순으로 나온다

1 작업반 개발	2 검토·릴리스	3 공개	4 인증	5 개정
회원사 전문가가 WG·JWG에서 초안 작성	회원 검토와 릴리스 후보(RC)를 거쳐 정식 릴리스	온라인 레퍼런스, NodeSet2 XML (UA-Nodeset)	CTT 시험, 독립 시험소 인증, UACTT Lite	현장 피드백 반영, 유지보수 릴리스(1.05.0x)

1.03 제품 인증 지원은 **2026년 말 종료** → 신규 제품·과제는 **1.05 기준**으로 준비
UAFX 1.00(2022-11)은 65개 이상 회원사, 320명 넘는 전문가가 4년간 개발

출처: OPC Foundation Compliance Corner, UAFX 보도자료

INDUSTRIAL AI RESEARCH CENTER

CONTENTS

I OPC UA 개념

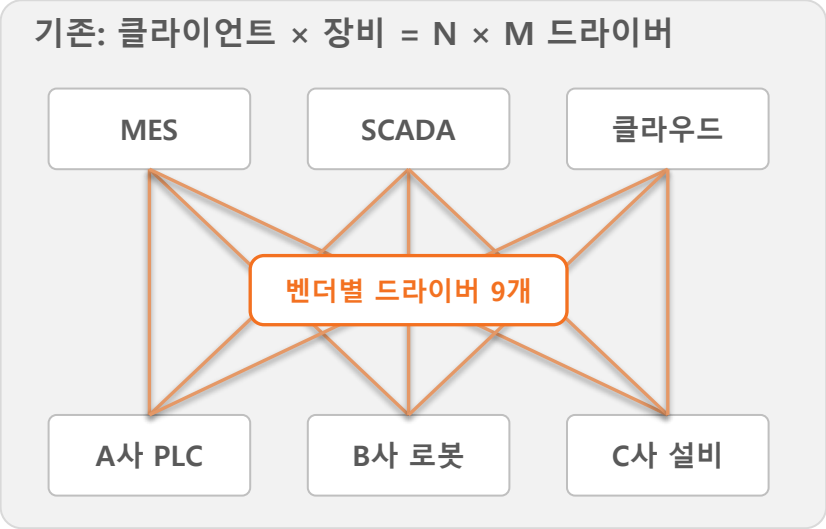
II OPC Foundation

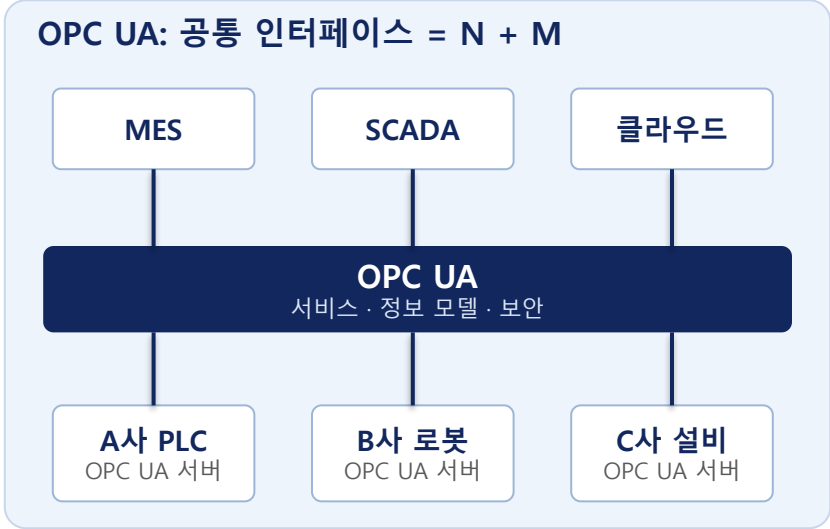
III 정보 모델링

IV 시스템 및 서비스

■ OPC UA 기반 상호운용성

➢ 벤더별 드라이버 대신, 같은 인터페이스와 같은 정보 모델로 연결한다





1 통신 상호운용성

같은 서비스 · 인코딩 · 보안 → Profile 선언 · CTT 인증으로 확인

2 의미 상호운용성

같은 정보 모델 → 협력 규격(DI · Machinery …)으로 노드 이름 · 타입 통일

근거: OPC 10000-1 §4.2 · OPC 10000-7 Profiles · UA Companion Specifications

정보 모델링

■ 개념

➢ 정보 모델링은 데이터에 타입·구조·관계를 붙이는 일이다

주소 기반 (Modbus)

주소	값
40001	372
40002	1
40003	1350

372는 온도인가? 단위는 0.1 °C인가?
→ 별도 문서(주소 맵)를 봐야 알 수 있다
→ 주소 맵이 바뀌면 상위 코드도 바뀐다



정보 모델 기반 (OPC UA)

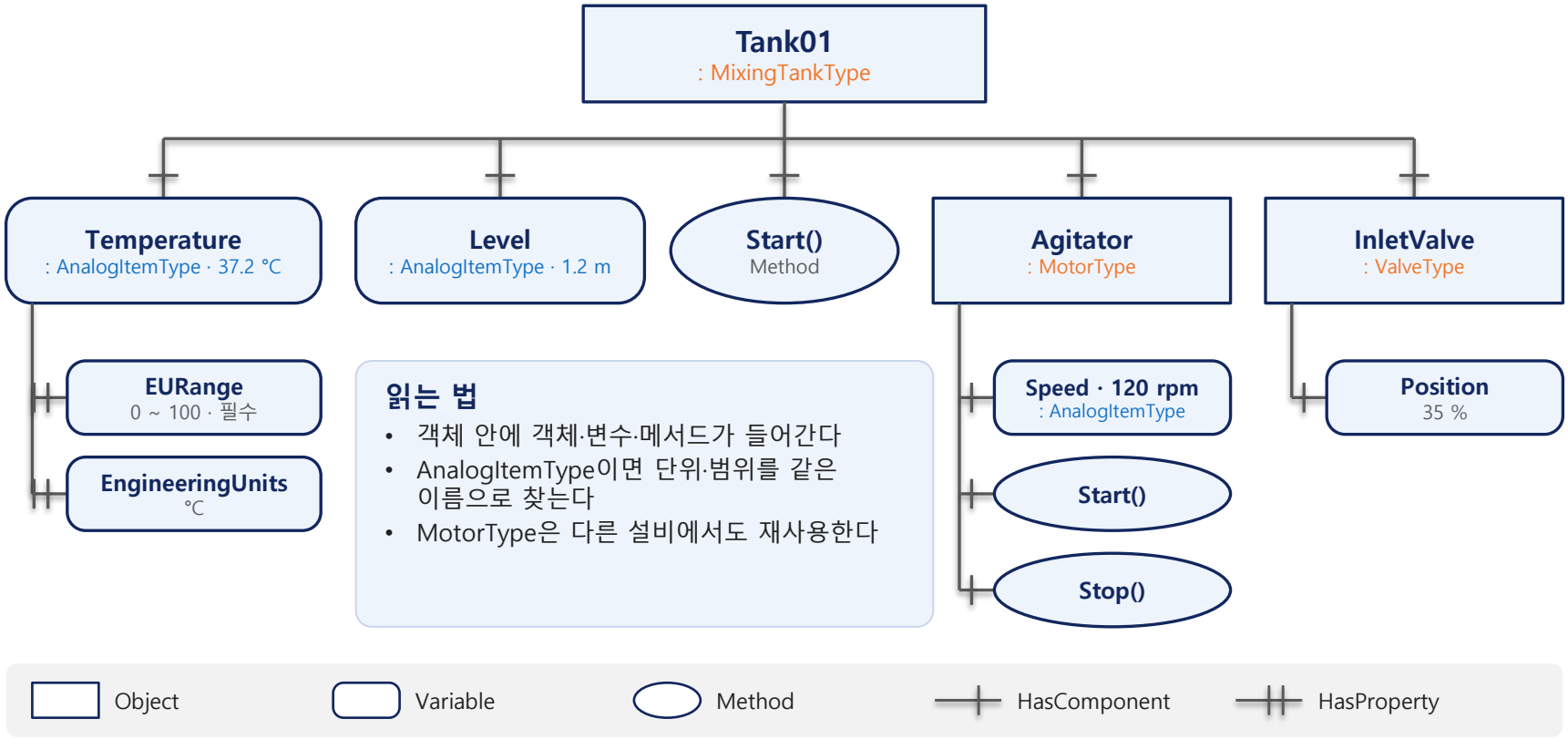
Plant
└ **Tank01** : MixingTankType
  ├ **Temperature** = 37.2
  | Double · °C · 범위 0~100
  | 품질 Good · 2026-10-01 10:15:02
  ├ **State** = Running
  └ **SerialNumber** = MT-2026-001

탐색만으로 이름·형식·단위·품질·시각을 안다

자기 기술(Self-describing) 데이터

■ OPC UA 정보 모델 예시

➢ 설비를 객체·변수·속성·메서드의 계층으로 표현하고, 표준 타입을 재사용한다



파랑 = 표준 타입 (Part 8) 주황 = 사용자 정의 타입

근거: OPC 10000-8 §5.3.2 AnalogItem VariableTypes · OPC 10000-3 §4.6

■ OPC UA 클라이언트에서의 활용

➢ 클라이언트는 정보 모델을 탐색해 의미를 알아내고, 타입 단위로 처리한다

① Browse: 구조 발견 → ② HasTypeDefinition: 타입 확인 → ③ 타입별 처리 자동화

범용 클라이언트

UaExpert 등

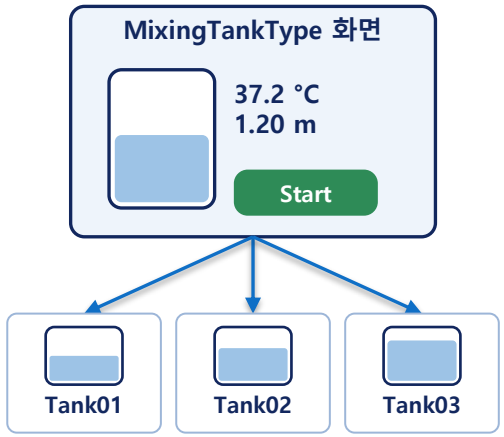
Address Space

- ▼ Objects
  - ▼ Plant
    - ▼ Tank01 : MixingTankType
      - **Temperature = 37.2**
      - Level = 1.20
      - Start()

속성	값
DataType	Double
EngineeringUnits	°C
EURange	0 ~ 100

HMI 화면 템플릿

타입 하나 → 화면 하나



인스턴스마다 자동 연결
탱크가 늘어도 화면 작업 없음

MES · 분석

타입으로 대상 검색

```
# 타입으로 대상 찾기
tanks = find(Objects,
       MixingTankType)
for t in tanks:
    subscribe(t.Temperature)
    subscribe(t.Level)
```

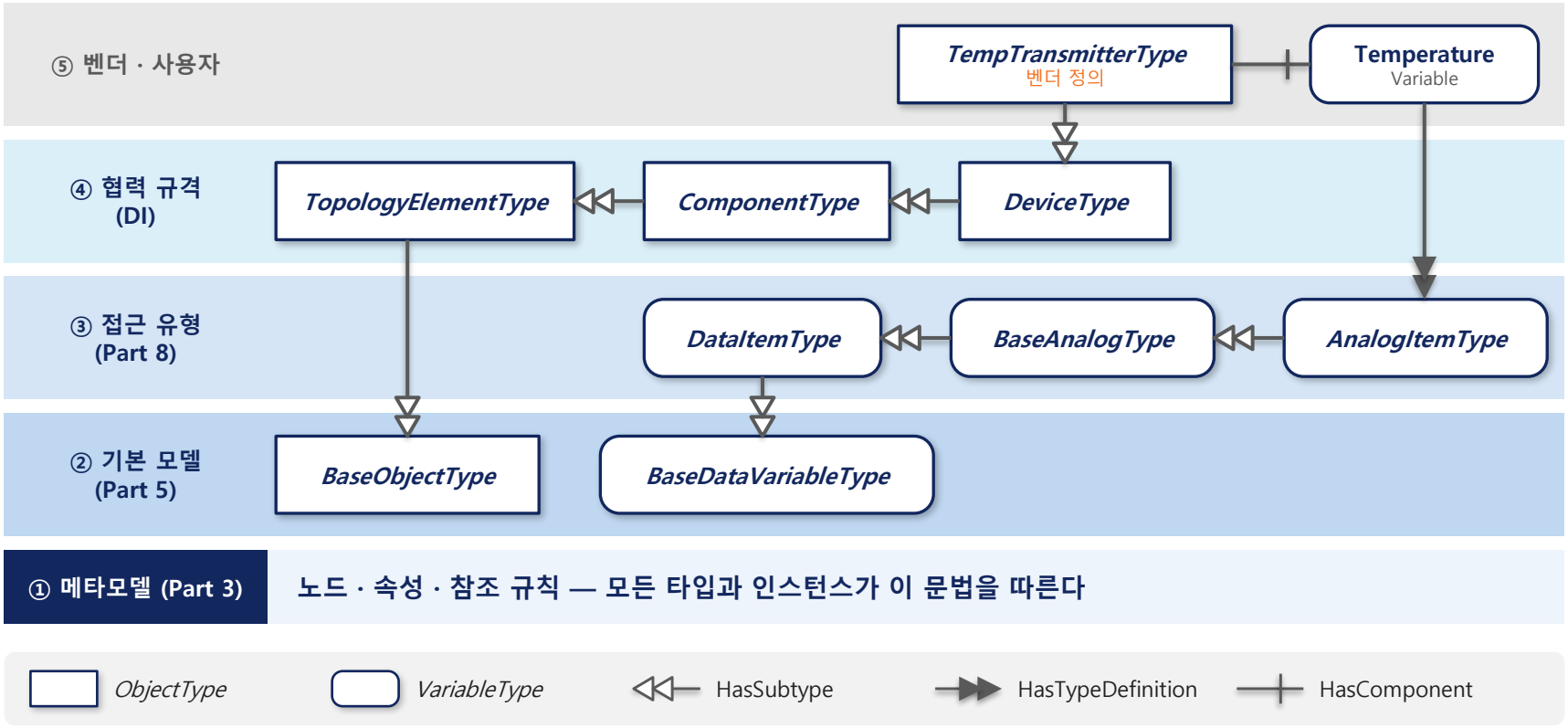
- Tank04를 추가해도 코드는 그대로
- NodeId 대신 타입이 정한 이름(BrowseName)으로 접근
- 협력 규격 타입이면 벤더가 달라도 같은 코드

근거: OPC 10000-4 §5.9 View Service Set · OPC 10000-3 §4.6.1

INDUSTRIAL AI RESEARCH CENTER

■ OPC UA 정보 모델 계층

➢ 위 계층의 타입은 아래 계층의 타입을 상속해 만든다

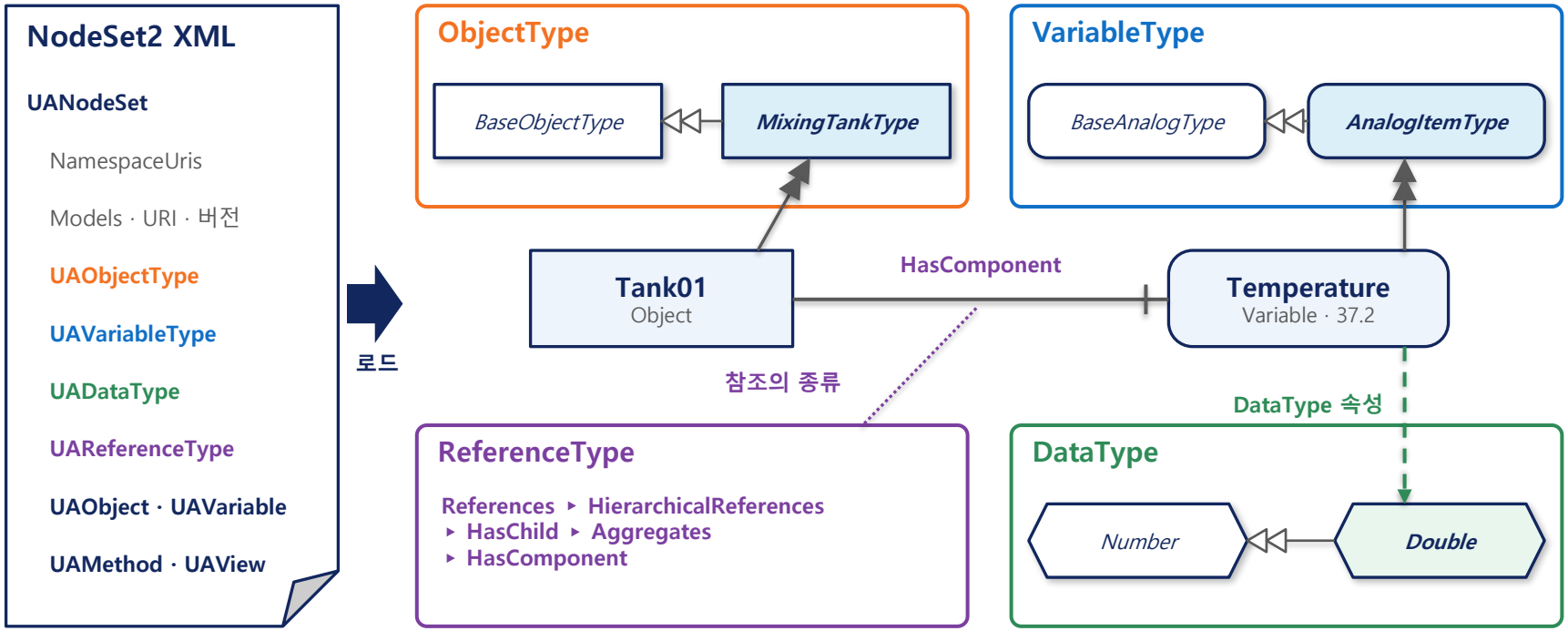


근거: OPC 10000-100 DI §4.6·4.7 · OPC 10000-8 §5.3.2 · OPC 10000-5

정보 모델링

■ OPC UA 정보 모델 상세 구조

➢ 정보 모델은 네 종류의 타입 정의와 인스턴스로 이루어진다



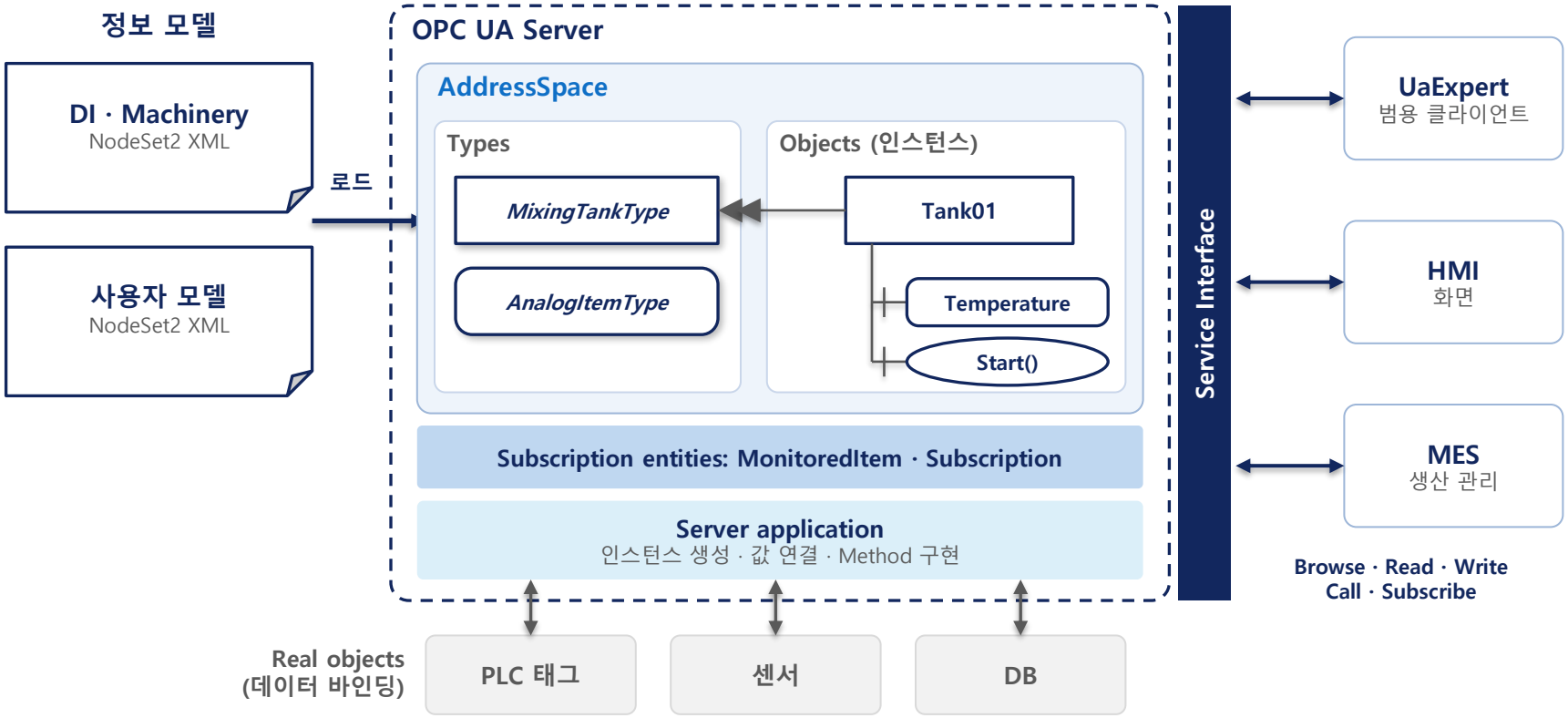
정보 모델 = 네임스페이스(URI) 하나에 담긴 **타입 정의 + 인스턴스** → NodeSet2 XML로 배포·교환

근거: OPC 10000-3 §4.6 TypeDefinitionNodes · OPC 10000-6 Annex F (NodeSet XML) · UA-Nodeset

정보 모델링

■ OPC UA 서버 아키텍처

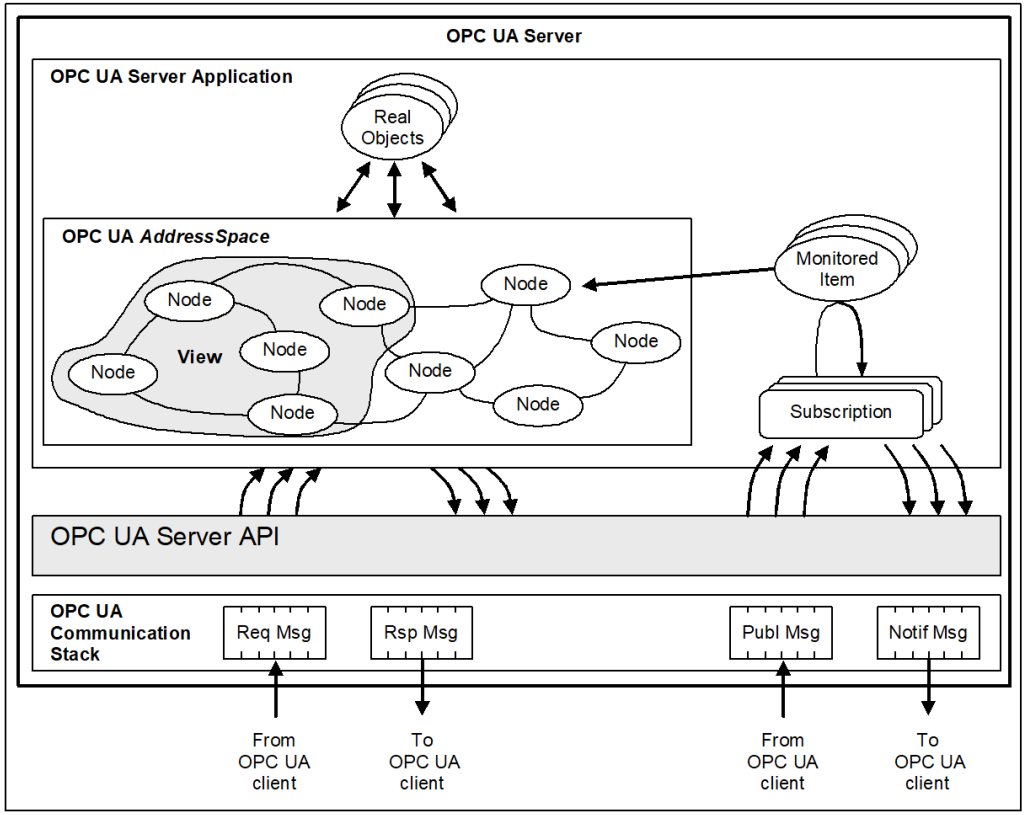
➢ 서버는 정보 모델을 주소공간에 올리고, 실제 객체와 연결해 서비스로 제공한다



원문: OPC 10000-1 Figure 4 – OPC UA Server architecture · §5.3.4 OPC UA AddressSpace

■ OPC UA 서버 아키텍처

➢ 서버는 정보 모델을 주소공간에 올리고, 실제 객체와 연결해 서비스로 제공한다



■ OPC UA 객체 모델

➢ 객체 모델의 각 요소는 주소공간에서 노드와 참조로 표현된다

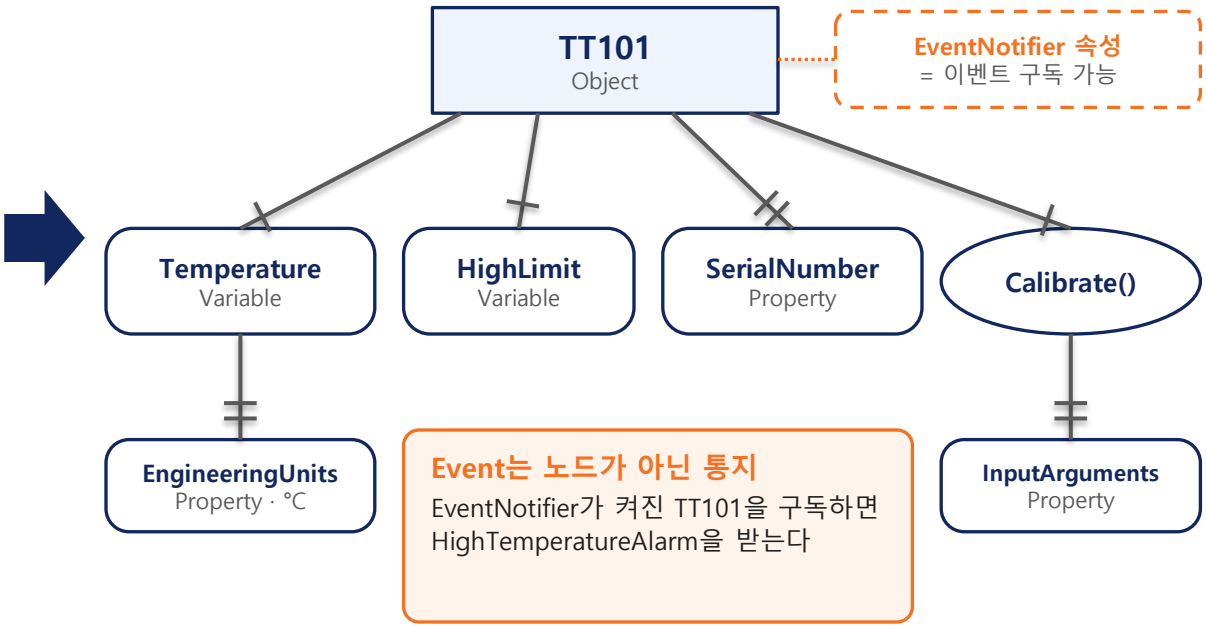
TT101 (Object)

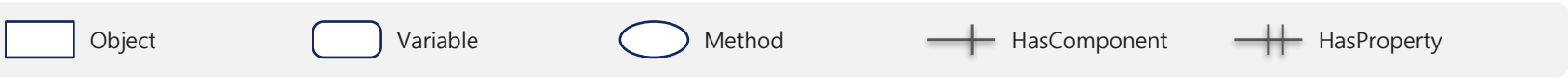
Variables
Temperature · HighLimit
SerialNumber (Property)

Methods
Calibrate()

Events
HighTemperatureAlarm

I 장의 객체 모델 (개념)



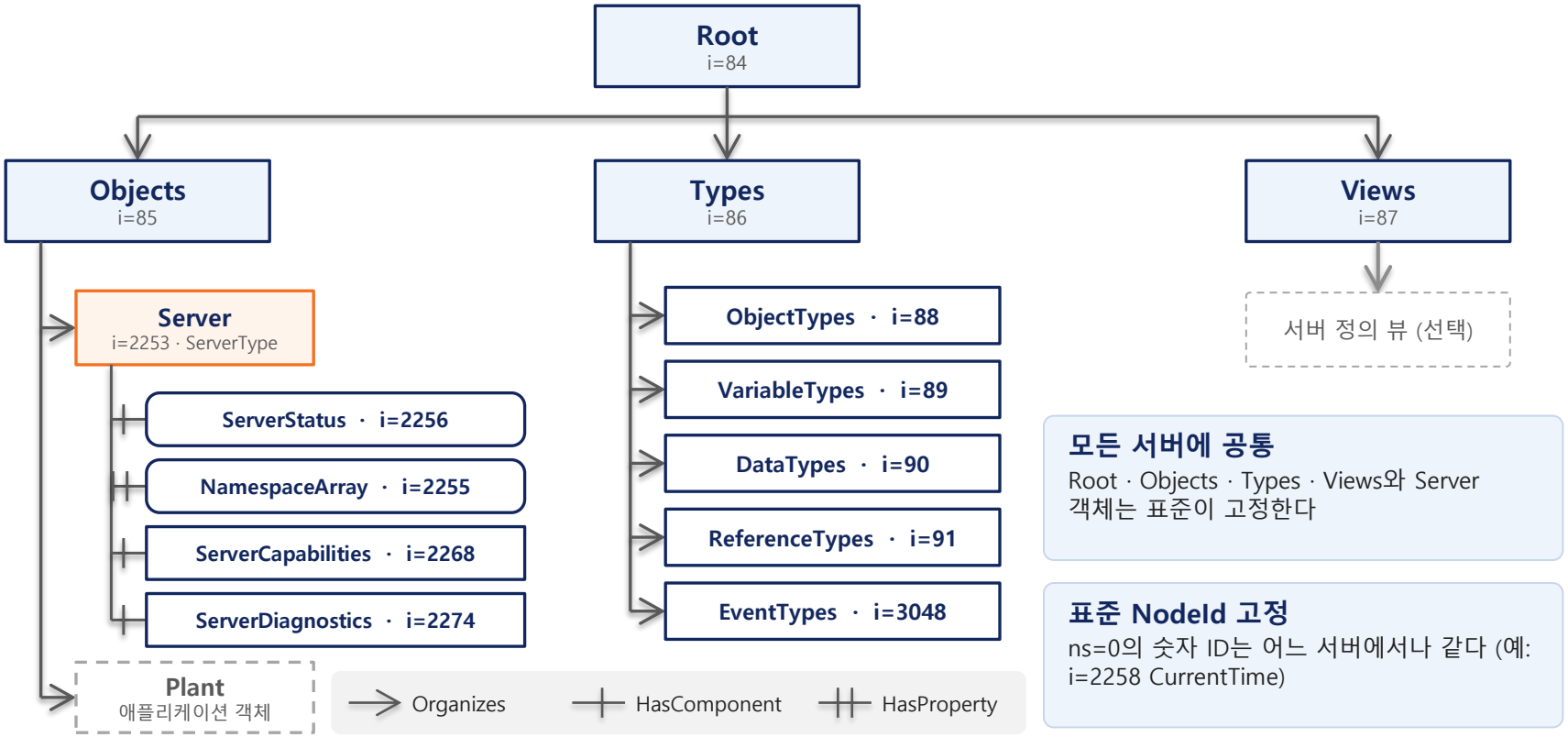


원문: OPC 10000-3 Figure 2 – OPC UA Object Model · §5.5 Objects (EventNotifier)

정보 모델링

■ 어드레스 스페이스 모델의 표준 노드 구조

➢ 모든 서버는 Root 아래 Objects·Types·Views라는 같은 뼈대를 갖는다



모든 서버에 공통
Root · Objects · Types · Views와 Server 객체는 표준이 고정한다

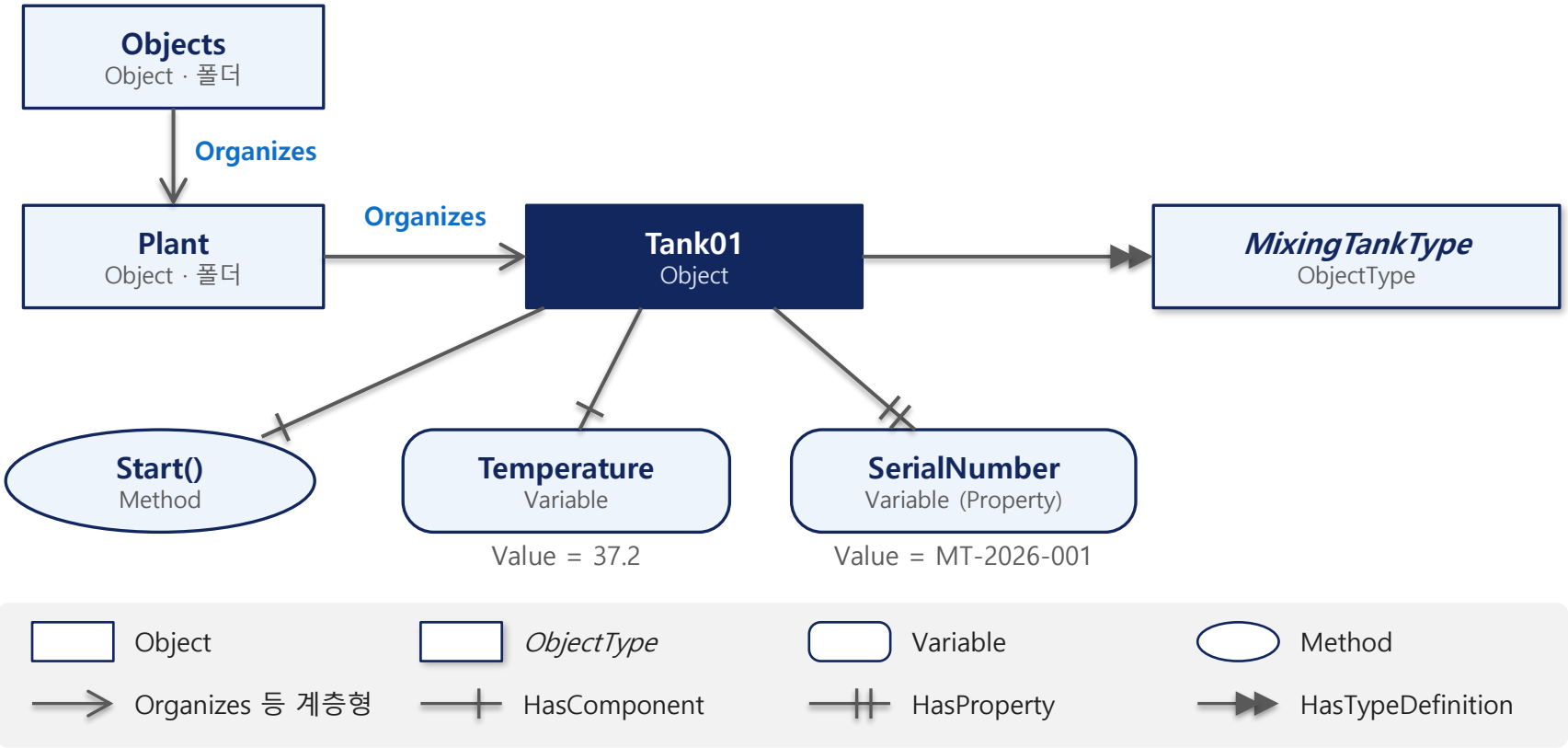
표준 NodeId 고정
ns=0의 숫자 ID는 어느 서버에서나 같다 (예: i=2258 CurrentTime)

원문: OPC 10000-5 Figure 1 – Standard AddressSpace structure · §8.3.2 Server Object

INDUSTRIAL AI RESEARCH CENTER

■ 어드레스 스페이스 모델

➢ 주소공간은 노드와 참조로 이루어진 그래프다



원문: OPC 10000-3 Figure 2 – Object Model · Figure 7 – Object and its Components · 표기법: Annex C.2.2

INDUSTRIAL AI RESEARCH CENTER

■ 어드레스 스페이스 노드 모델

➢ 노드는 NodeClass가 정한 속성과, 다른 노드를 가리키는 참조로 이루어진다

Attributes
- 노드를 설명하는 데이터
- NodeClass마다 정해져 있고 추가할 수 없다
- Read · Write · Subscribe로 접근

References
- 다른 노드와의 관계
- 참조 타입 + 대상 노드
- 속성처럼 노드의 기본 구성 요소

Temperature
NodeClass = Variable

Attributes

속성	값
NodeId	ns=2;i=12
BrowseName	2:Temperature
DisplayName	Temperature
Value	37.2
DataType	Double
AccessLevel	CurrentRead

References

참조 타입	대상 노드
HasTypeDefinition	→ AnalogItemType
HasProperty	→ EURange
HasProperty	→ EngineeringUnits
ComponentOf	← Tank01 (역방향)

노드 = 속성 + 참조
객체와 구성 요소는 속성으로 기술되고 참조로 연결된 노드의 집합이다 (§4.4.1)

AnalogItemType
VariableType

EURange
Property · 0 ~ 100

EngineeringUnits
Property · °C

HasComponent

Tank01
Object

원문: OPC 10000-3 Figure 3 – AddressSpace Node Model · §4.4.3 Attributes

INDUSTRIAL AI RESEARCH CENTER

■ 어드레스 스페이스 모델

➢ NodeClass는 인스턴스 4종과 타입 4종으로 나뉜다

인스턴스 NodeClass		타입 NodeClass
Object 설비·폴더처럼 구조를 이루는 실체	HasTypeDefinition →	**ObjectType** 객체의 구성 요소와 구조를 정의
Variable 값을 가진 노드 (DataVariable, Property)	HasTypeDefinition →	**VariableType** 변수의 구조와 기본값을 정의
Method 클라이언트가 호출하는 기능	DataType 속성 →	**DataType** 값의 데이터형 (Double, String, 구조체 ...)
View 주소공간의 일부만 보여 주는 부분 집합		**ReferenceType** 참조(관계)의 종류와 의미를 정의

모든 참조는 ReferenceType의 한 종류이고, 인스턴스는 HasTypeDefinition으로 자기 타입을 가리킨다

근거: OPC 10000-3 §5 Standard NodeClasses · NodeClass별 기호: Annex C.2.2 Table C.1

■ 어드레스 스페이스 모델

➢ 모든 노드는 NodeId로 식별되고 속성으로 기술된다

구분	속성	예시 값
공통	NodeId	ns=2;i=12
	NodeClass	Variable
	BrowseName	2:Temperature
	DisplayName	Temperature
Variable	Value	37.2
	DataType	Double
	ValueRank	-1 (스칼라)
	AccessLevel	CurrentRead
	MinimumSamplingInterval	100 ms

Temperature 변수의 예 · 속성 종류는 NodeClass마다 정해져 있다

NodeId 형식

ns=2;s=Tank01.Temperature

- **i** **숫자** · ns=0;i=85 (Objects 폴더)
- **s** **문자열** · ns=2;s=Tank01.Temperature
- **g** **GUID** · ns=2;g=...
- **b** **바이트열** · ns=2;b=...

네임스페이스 배열 (NamespaceArray)

인덱스	URI
0	http://opcfoundation.org/UA/
1	서버 로컬 URI
2	urn:example:mixingplant

인덱스는 바뀔 수 있다 → **URI로 인덱스 조회**

원문: OPC 10000-3 Figure 3 – AddressSpace Node Model · NodeId 문자열: OPC 10000-6 §5.1.12

INDUSTRIAL AI RESEARCH CENTER

■ 어드레스 스페이스 참조 모델

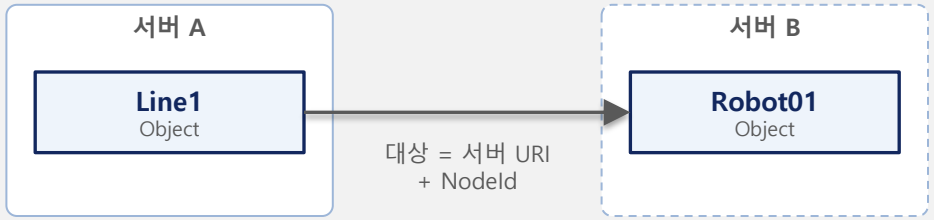
➢ 참조는 출발 노드·참조 타입·대상 노드의 조합으로 식별되는 방향 있는 관계다





세 값의 조합으로 참조를 유일하게 식별한다 (§4.4.4)

원격 참조



HasComponent = ReferenceType 노드

속성	값
IsAbstract	False
Symmetric	False
InverseName	ComponentOf
상위 타입	Aggregates

대칭 참조 (Symmetric = True)



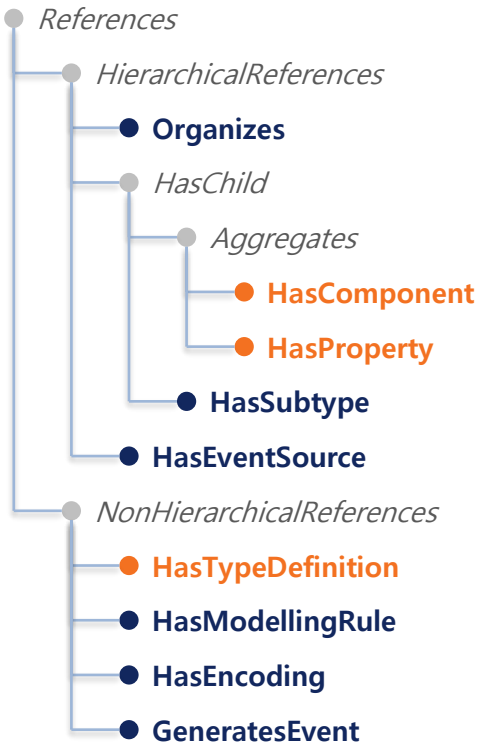
Browse 방향: Forward · Inverse · Both
+ 참조 타입 필터로 필요한 관계만 따라간다

원문: OPC 10000-3 Figure 4 – Reference Model · §5.3 ReferenceType NodeClass · Part 5 §11.7

INDUSTRIAL AI RESEARCH CENTER

■ 어드레스 스페이스 모델

➢ 참조 타입 계층과 타입 정의가 노드의 의미를 만든다



회색 기울임 = 추상 타입 · 주황 = 자주 쓰는 참조



원문: OPC 10000-3 Figure 38 – Standard ReferenceType Hierarchy

■ 표준 정보 모델

➢ 접근 유형별 모델이 계측값·알람·이력·프로그램 표현을 표준화한다

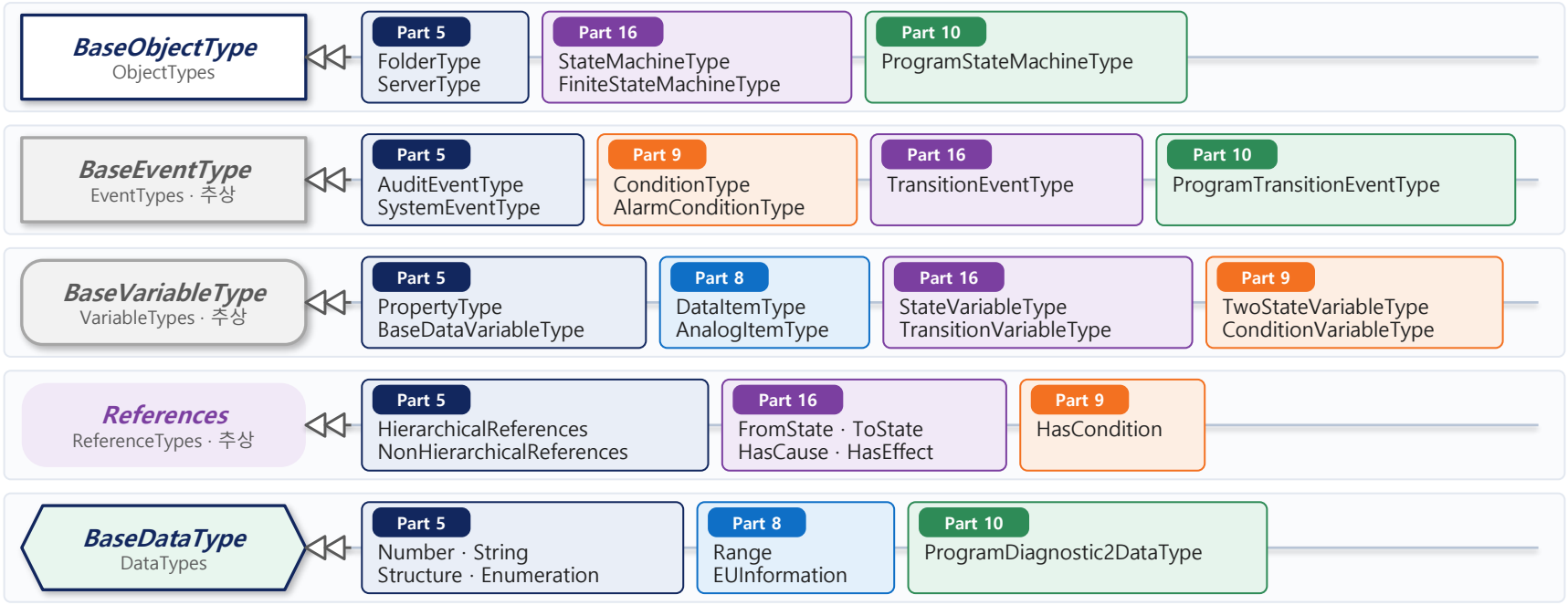
Part	모델	대표 타입	무엇을 표준화하나
8	데이터 접근(DA)	AnalogItemType TwoStateDiscreteType	계측값의 범위(EURange)·단위
9	알람·컨디션(A&C)	AlarmConditionType LimitAlarmType	경보 상태·심각도·확인(Ack)
10	프로그램	ProgramStateMachineType	장시간 작업의 시작·일시정지·중단
11	이력 접근(HA)	HistoricalDataConfigurationType	과거 값·이벤트 저장과 조회
16	상태 기계	FiniteStateMachineType	설비 운전 상태와 상태 전이
100	디바이스(DI)	DeviceType TopologyElementType	기기 식별·파라미터·진단 그룹

근거: OPC 10000-8, -9, -10, -11, -16, -100

INDUSTRIAL AI RESEARCH CENTER

■ OPC UA 정보 모델 상세 구조

➢ 표준 정보 모델은 Part 5의 기본 타입 트리에 각 Part가 하위 타입을 더한 구조다



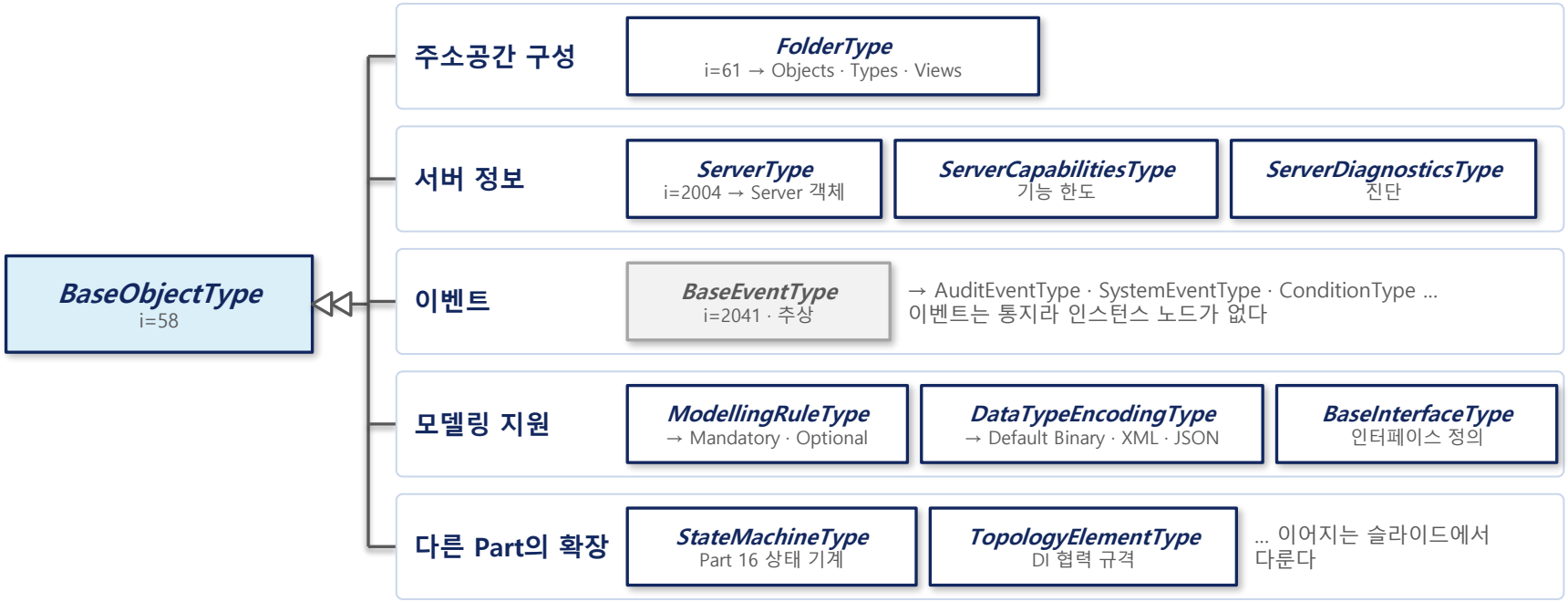
모든 표준 타입은 **네임스페이스 0**에 있다 · 협력 규격·벤더 타입도 같은 트리 아래 **하위 타입**으로 붙는다

근거: OPC 10000-5 §6·§7 · -8 §5.1 · -9 §5.1 · -10 §5.1 · -16 §4.4.1

INDUSTRIAL AI RESEARCH CENTER

■ 표준 객체 타입 구조

➢ 모든 ObjectType은 BaseObjectType에서 시작해 용도별로 갈라진다

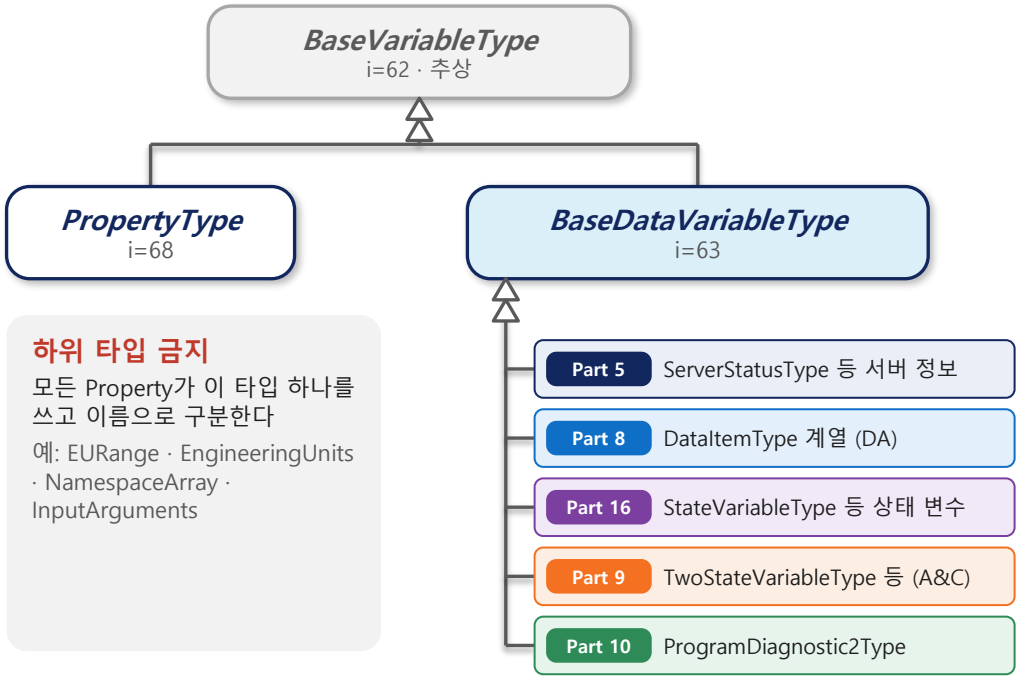


근거: OPC 10000-5 §6 Standard ObjectTypes · §6.2 BaseObjectType

INDUSTRIAL AI RESEARCH CENTER

■ 표준 변수 타입 구조

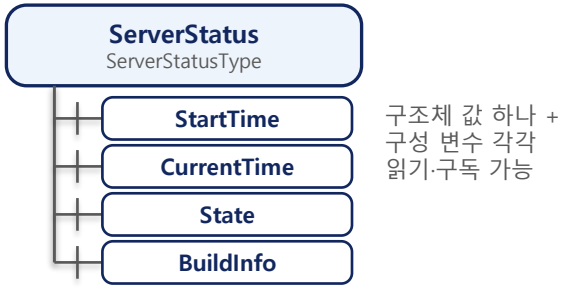
➢ VariableType은 Property와 DataVariable 두 갈래로 나뉜다

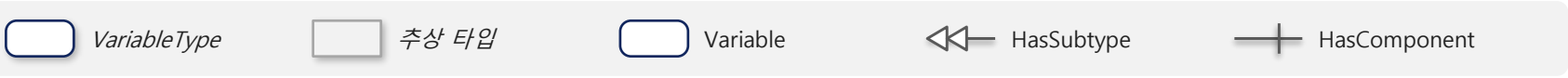


Property vs DataVariable

	Property	DataVariable
역할	노드의 특성	객체의 데이터(값)
연결	HasProperty	HasComponent
타입	PropertyType만	BaseDataVariableType 계열
하위 노드	없음 (leaf)	가질 수 있음 (복합)
예	EURange	Temperature

복합 DataVariable 예 (Server 객체)

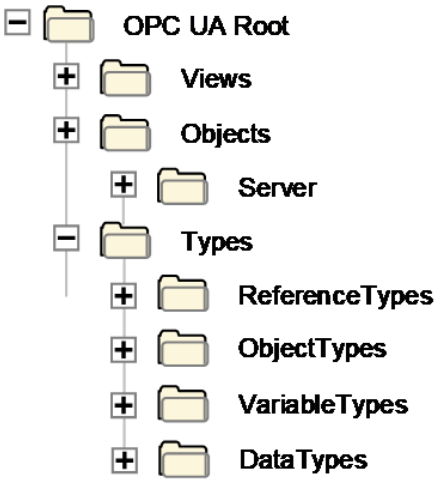




근거: OPC 10000-5 §7 Standard VariableTypes · OPC 10000-3 §5.6 Variables

■ 어드레스 스페이스 구조 표준화

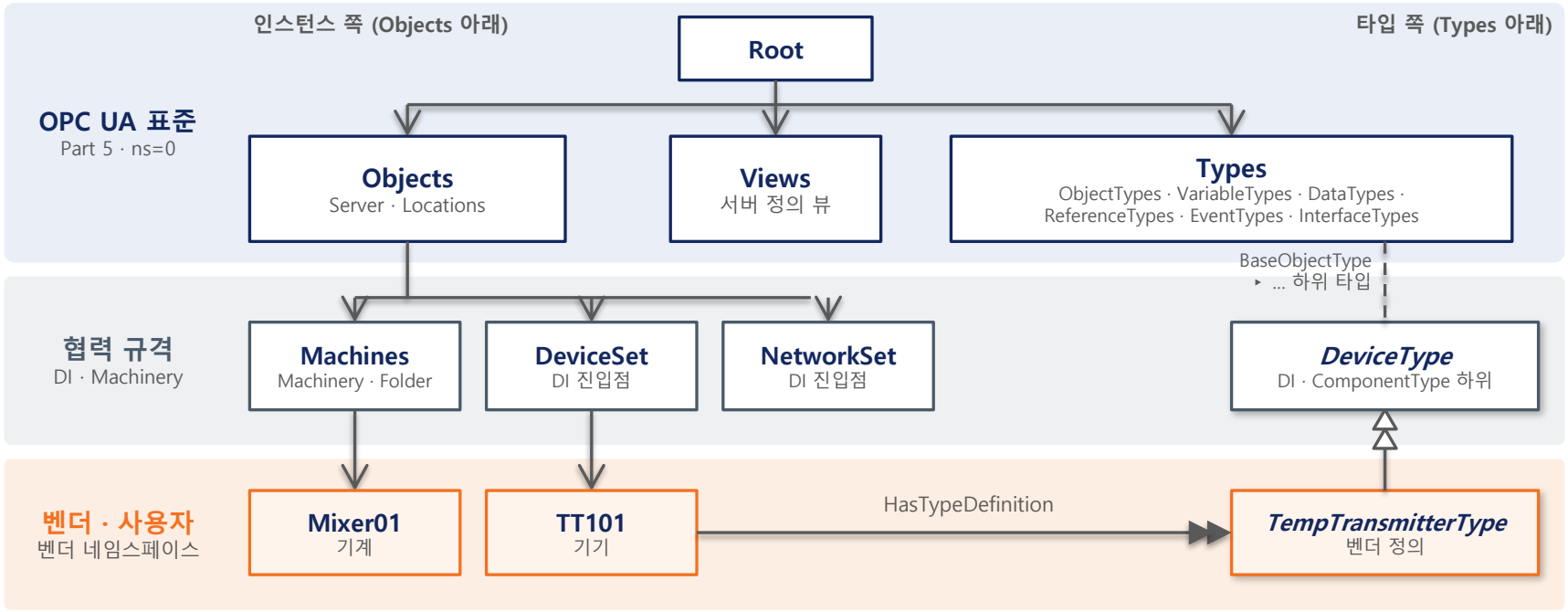
➢ 표준과 협력 규격이 진입점을 정해 두어, 어느 서버든 같은 경로로 찾아 들어간다



정보 모델링

■ 어드레스 스페이스 구조 표준화

➢ 표준과 협력 규격이 진입점을 정해 두어, 어느 서버든 같은 경로로 찾아 들어간다



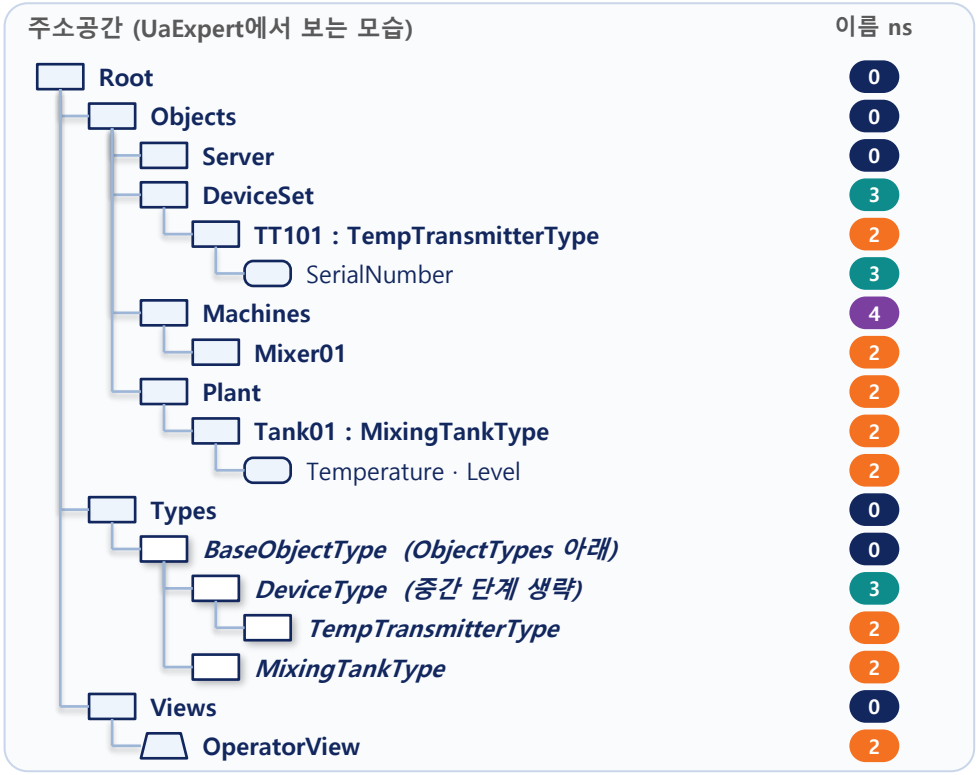
인스턴스는 **Objects 아래 진입점**(DeviceSet · Machines)에, 타입은 **BaseObjectType의 하위 타입**으로 → 범용 클라이언트가 같은 경로로 찾는다

원문: OPC 10000-5 Figure 1 – Standard AddressSpace structure · DI §4.9 DeviceSet · Machinery §9.2 Machines

INDUSTRIAL AI RESEARCH CENTER

■ 어드레스 스페이스 구조 예시

➢ 서버 하나에 표준·협력 규격·벤더 노드가 네임스페이스로 나뉘어 들어간다



NamespaceArray

ns	URI	정의
0	http://opcfoundation.org/UA/	OPC UA 표준
1	urn:mixingplant:server	서버 로컬
2	urn:example:mixingplant	벤더 모델
3	http://opcfoundation.org/UA/DI/	DI
4	http://opcfoundation.org/UA/Machinery/	Machinery

NodeId 예

Server → i=2253 (ns=0)
TT101 → ns=2;s=TT101
SerialNumber → ns=2;s=TT101.SerialNumber
이름(BrowseName)은 3:SerialNumber

Object *ObjectType* Variable View

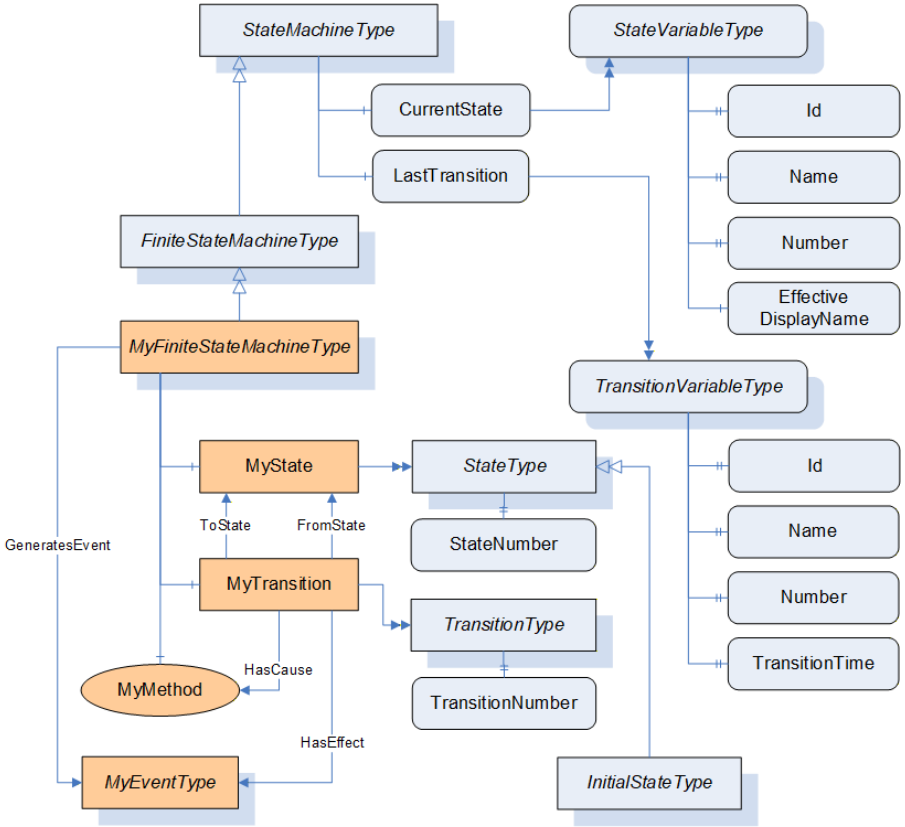
2 이후 번호는 서버마다 다를 수 있다
→ 클라이언트는 URI로 번호를 찾아 NodeId를 만든다

근거: OPC 10000-5 §8.2 AddressSpace 구조 · §8.3.2 Server Object (NamespaceArray) · DI §4.9

정보 모델링

■ 상태 기계 기본 정보 모델

➢ 상태 기계는 현재 상태 변수와, 타입에 정의한 상태·전이 객체로 표현한다

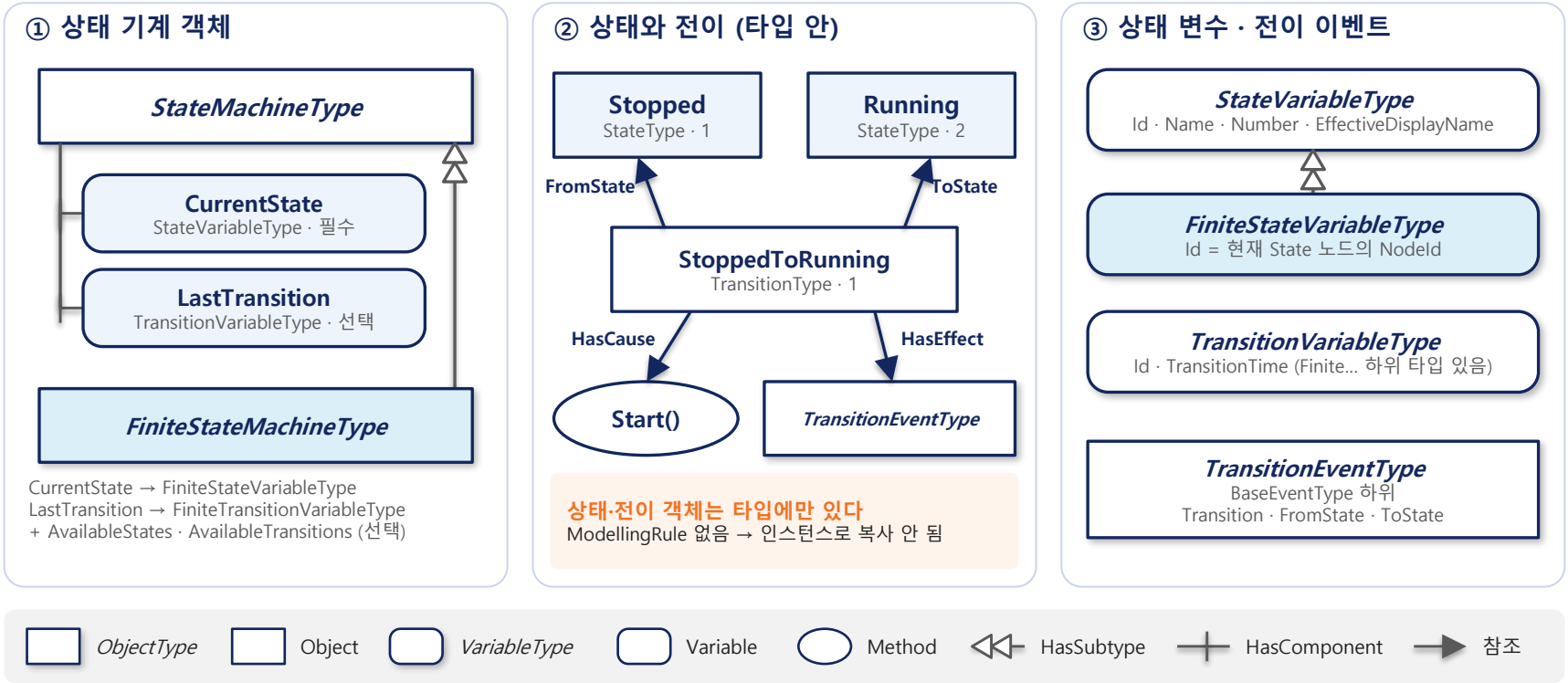


원문: OPC 10000-16 Figure 3 – The StateMachine Information Model · §4.4.6 FiniteStateMachineType

INDUSTRIAL AI RESEARCH CENTER

■ 상태 기계 기본 정보 모델

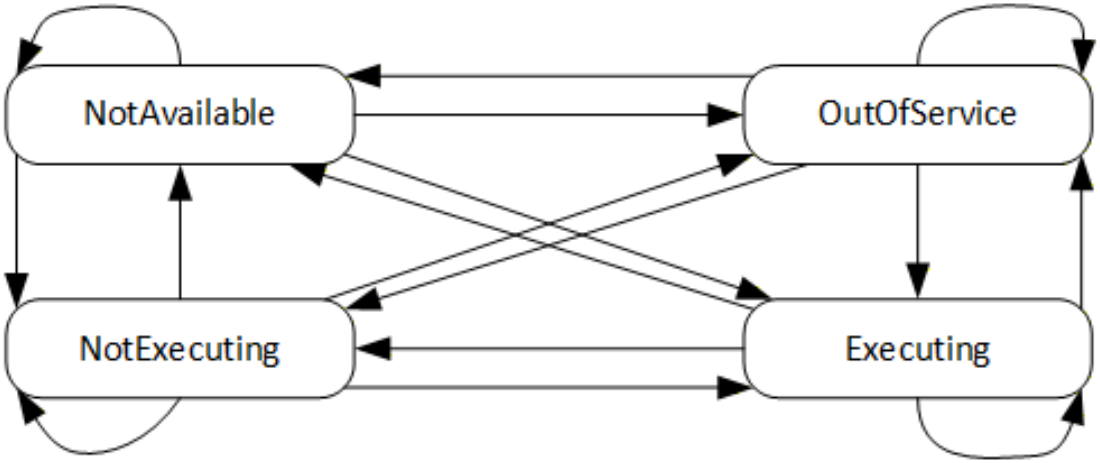
➢ 상태 기계는 현재 상태 변수와, 타입에 정의한 상태·전이 객체로 표현한다



원문: OPC 10000-16 Figure 3 – The StateMachine Information Model · §4.4.6 FiniteStateMachineType

■ 시스템 상태의 상태 기계 정보 모델

➢ 설비의 운전 상태도 FiniteStateMachineType의 하위 타입으로 표준화한다



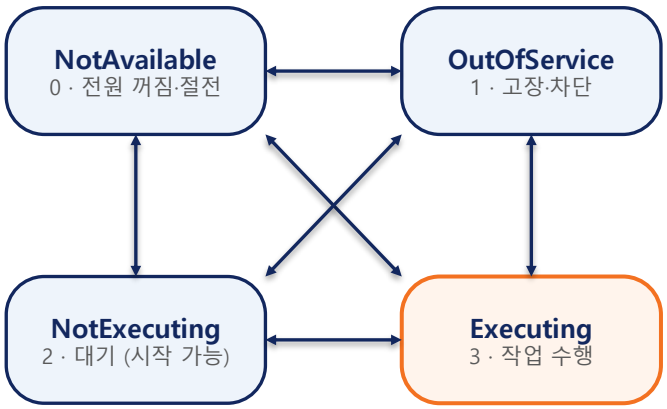
원문: OPC 40001-1 Figure 14 – MachineryItemState StateMachine · §12.2 · OPC 10000-16 §4.4.6

INDUSTRIAL AI RESEARCH CENTER

■ 시스템 상태의 상태 기계 정보 모델

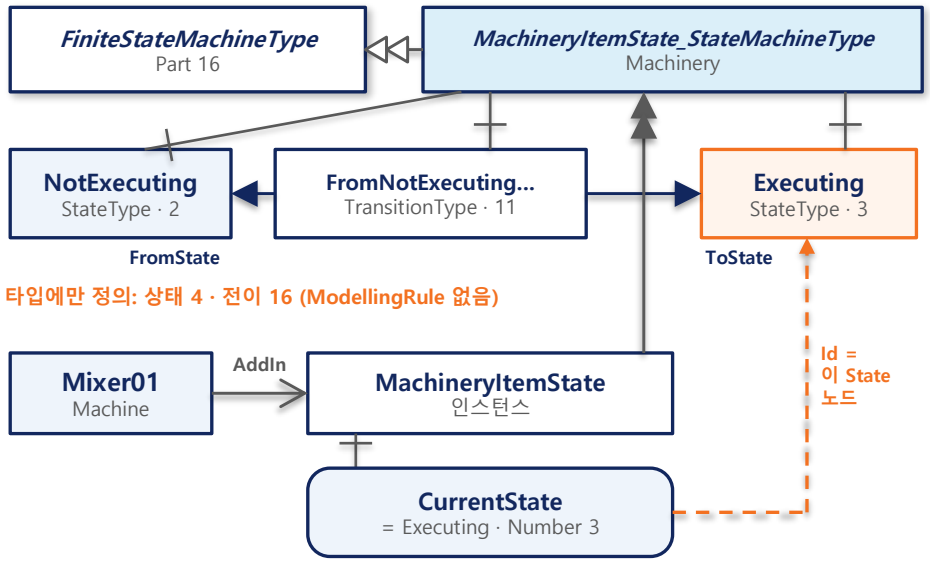
➢ 설비의 운전 상태도 FiniteStateMachineType의 하위 타입으로 표준화한다

운전 상태 네 가지 (Machinery)



모든 상태 쌍 사이에 전이 (자기 전이 포함 16개)
초기 상태는 벤더가 정한다 · 하위 상태 추가 가능

정보 모델: 타입 → 인스턴스

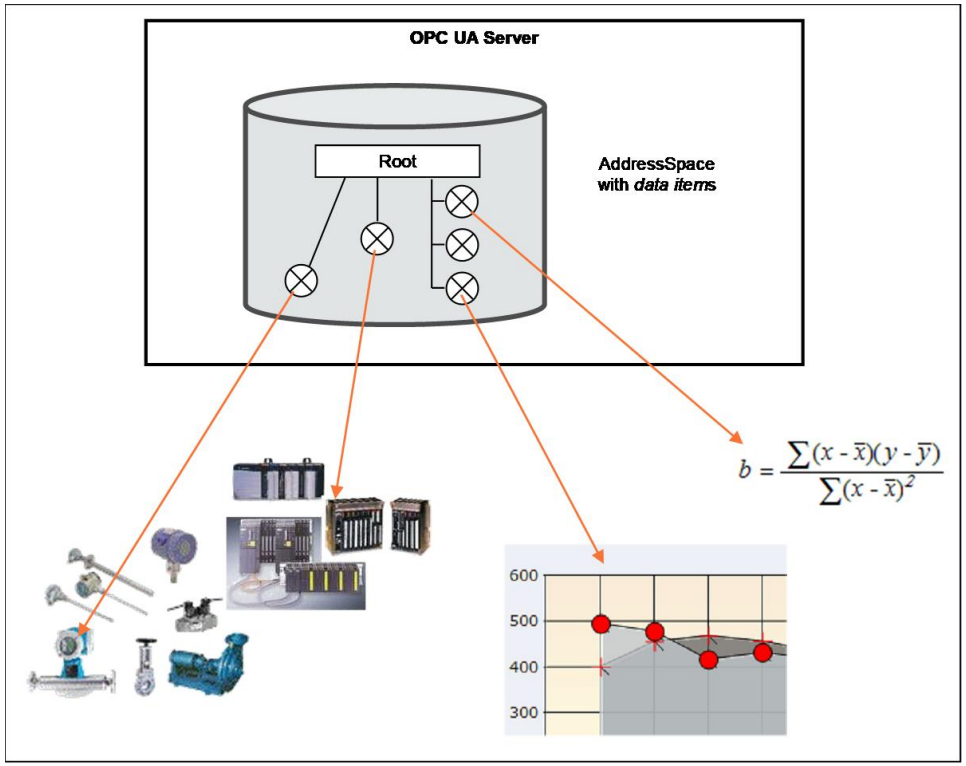


ObjectType | Object | Variable | HasSubtype | HasTypeDefinition | HasComponent | 참조

원문: OPC 40001-1 Figure 14 – MachineryItemState StateMachine · §12.2 · OPC 10000-16 §4.4.6

■ OPC UA 서버와 자동화 데이터

➢ Data Access (DA)는 서버 내 자동화 데이터의 표현 및 사용을 다룸

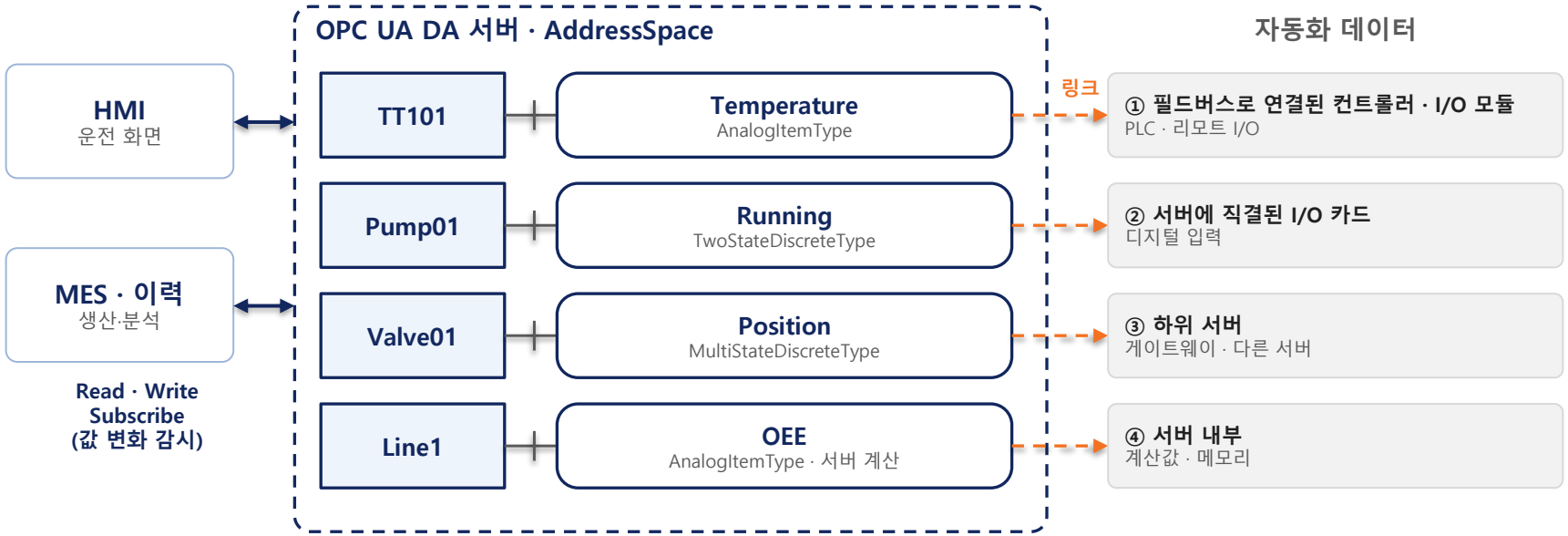


원문: OPC 10000-8 Figure 1 – OPC DataItems are linked to automation data · §5.4 Address Space model

INDUSTRIAL AI RESEARCH CENTER

■ OPC UA 서버와 자동화 데이터

➢ DA 서버는 곳곳의 자동화 데이터를 DataItem으로 연결해 투명하게 제공한다



DataItem = 자동화 데이터로 가는 링크 (§4)
항상 객체·폴더의 데이터 구성 요소로 정의된다 (§5.4)

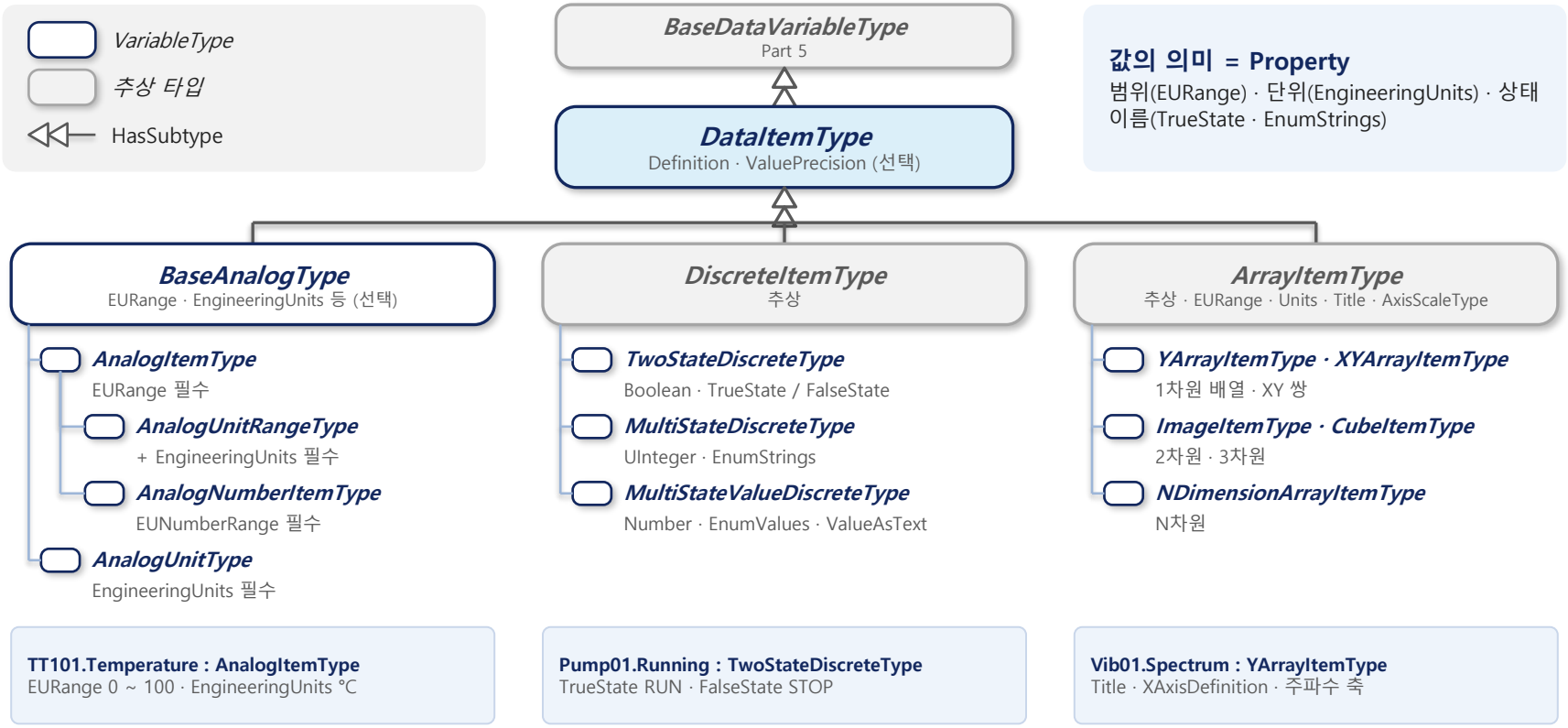
클라이언트가 받는 값 = DataValue
Value · StatusCode(품질) · SourceTimestamp · ServerTimestamp

원문: OPC 10000-8 Figure 1 – OPC DataItems are linked to automation data · §5.4 Address Space model

INDUSTRIAL AI RESEARCH CENTER

■ 데이터 접근 정보 모델

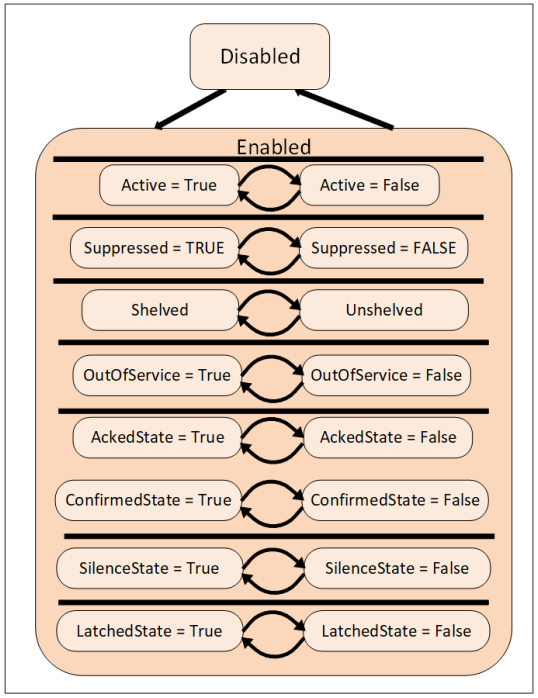
➢ DataItemType 아래 아날로그·이산·배열 타입이 값의 의미를 표준화한다



원문: OPC 10000-8 Figure 2 – DataItem VariableType hierarchy · §5.3 VariableTypes

■ 알람과 컨디션

➢ 알람(Alarm)은 '확인 가능한 상태(AcknowledgeableCondition)'를 특수한 형태로 확장한 것으로, 기존 상태(Condition)에 '활성(Active) 상태'를 비롯해 '보류(Shelving) 상태'나 '억제(Suppressed) 상태'와 같은 개념을 추가한 것



원문: OPC 10000-9 Figure 6 – Typical Alarm Timeline example · §4.2 Conditions · §4.10

INDUSTRIAL AI RESEARCH CENTER

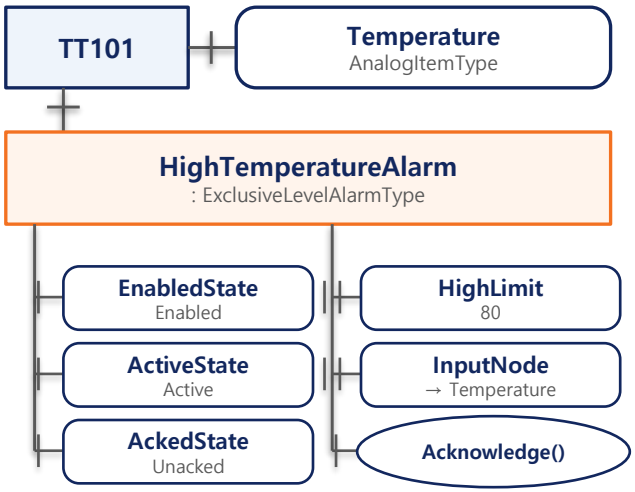
■ 알람과 컨디션

➢ 알람은 활성 상태를 가진 컨디션이고, 상태가 바뀔 때마다 이벤트로 알린다

HighTemperatureAlarm의 상태 변화



주소공간 표현 (§4.10)



노출은 선택 · 변화는 이벤트로 통지
재접속 시 ConditionRefresh로 동기화

Condition = 시스템·구성 요소의 상태 (§4.2)
EnabledState · Retain · Severity · Comment

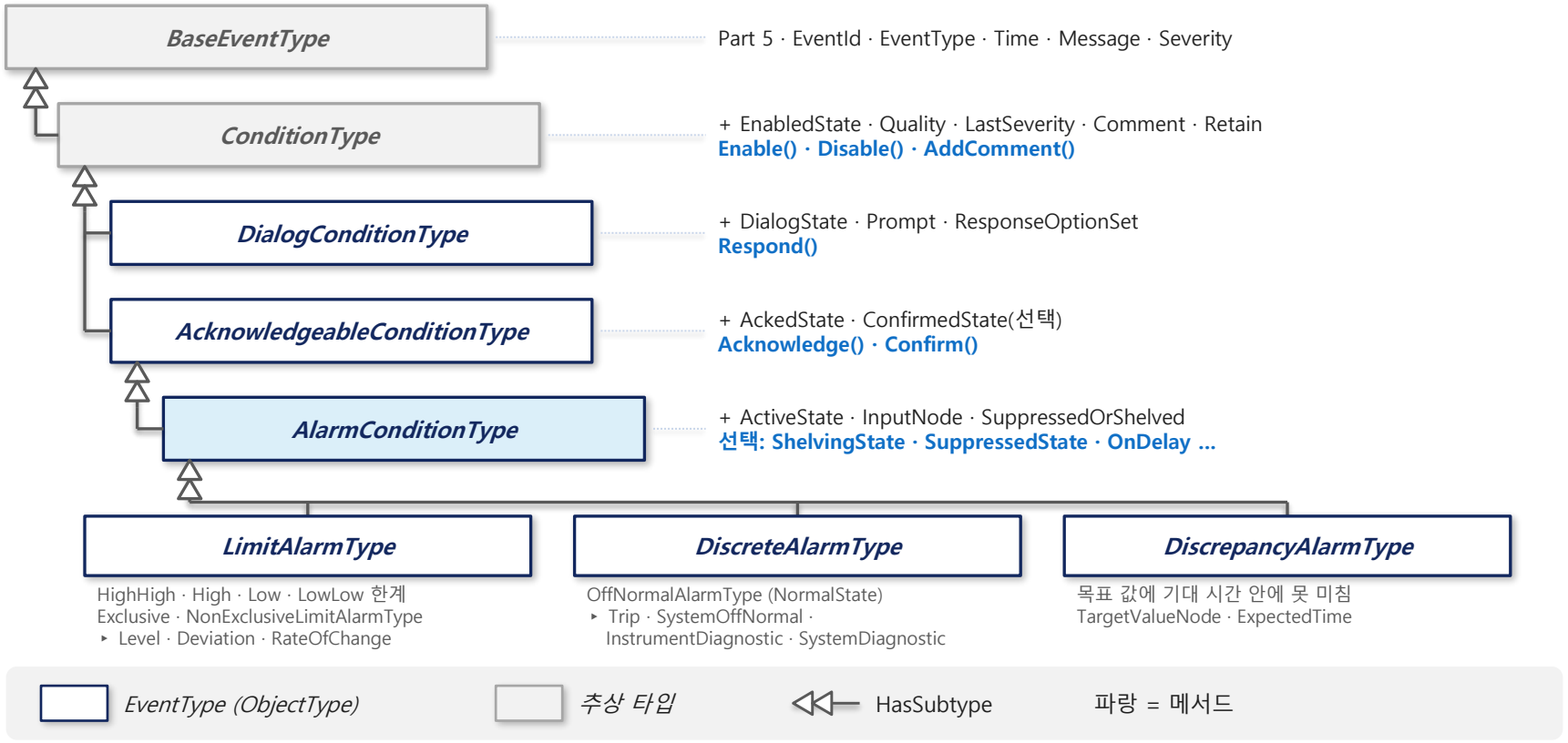
Alarm = 활성 상태를 더한 Condition (§4.8)
ActiveState · AckedState · Shelving · Suppression

원문: OPC 10000-9 Figure 6 – Typical Alarm Timeline example · §4.2 Conditions · §4.10

INDUSTRIAL AI RESEARCH CENTER

■ 컨디션 타입 계층 구조

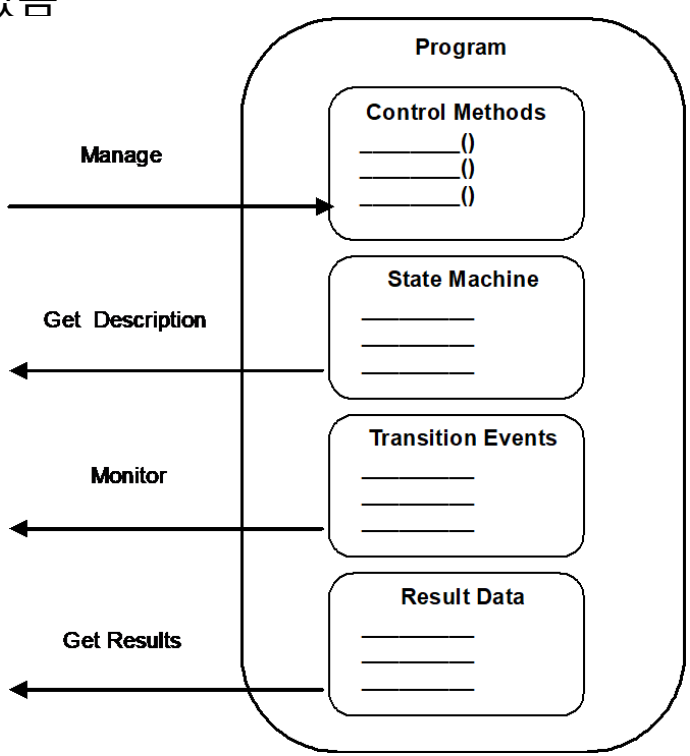
➢ 컨디션 타입은 이벤트 타입이며, 아래로 갈수록 상태와 메서드가 더해진다



원문: OPC 10000-9 Figure 8 – ConditionType hierarchy · Figure 14 – AlarmConditionType Hierarchy Model

■ 메소드와 프로그램 차이

➢ 프로그램은 서버나 기반 시스템 내에 존재하는 복합적인 기능으로, 클라이언트가 이를 호출하고 관리할 수 있음

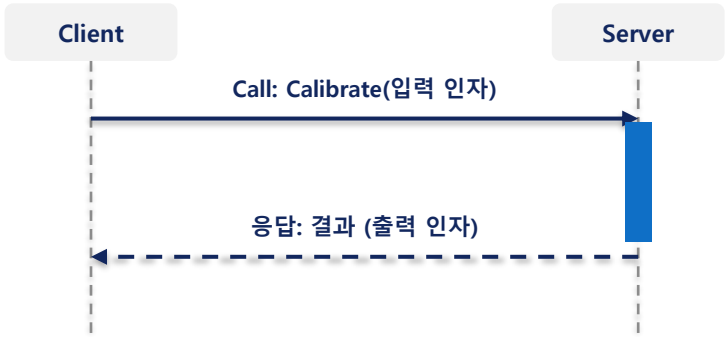


근거: OPC 10000-10 §4.1 General · Figure 2 – Program illustration

■ 메소드와 프로그램 차이

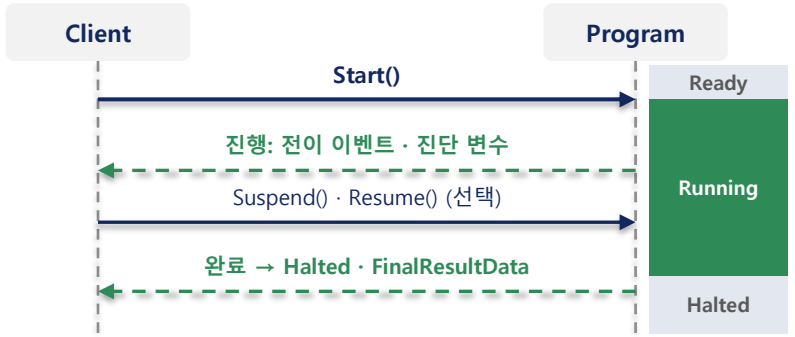
➢ Method는 한 번 호출로 끝나고, Program은 상태를 가지며 오래 실행된다

Method — 한 번의 호출



예: 계산 수행 · 카운터 리셋 (§4.1)

Program — 상태를 가진 실행



예: 배치 공정 · 파트 프로그램 · 도메인 다운로드 (§4.1)

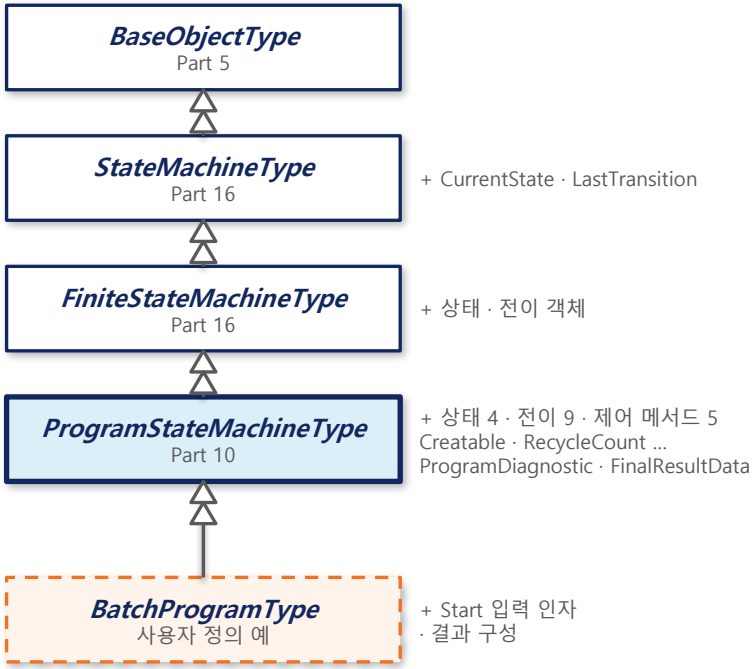
구분	Method	Program
주소공간	객체에 딸린 Method 노드	ProgramStateMachineType 객체 (상태 기계)
수명	호출 한 번으로 끝	Ready → Running → Halted, Reset으로 재사용
제어	Call 서비스	Start · Suspend · Resume · Halt · Reset
결과 · 감시	응답의 출력 인자	전이 이벤트 · 진단 변수 · FinalResultData

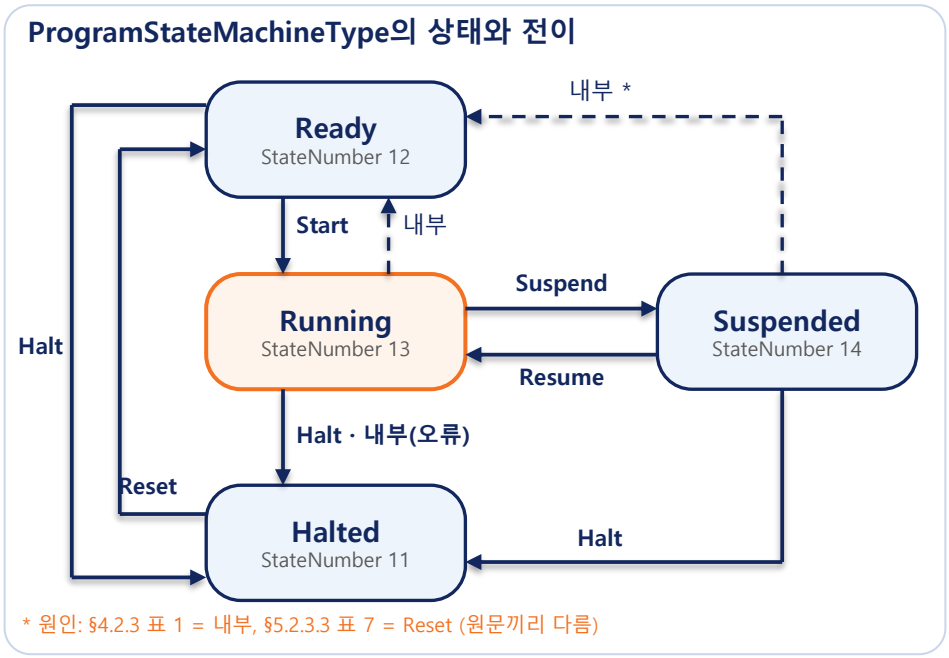
근거: OPC 10000-10 §4.1 General · Figure 2 – Program illustration

INDUSTRIAL AI RESEARCH CENTER

■ 프로그램 상태 기계 타입 계층 구조

➢ ProgramStateMachineType은 네 상태·아홉 전이·다섯 제어 메서드를 정한다





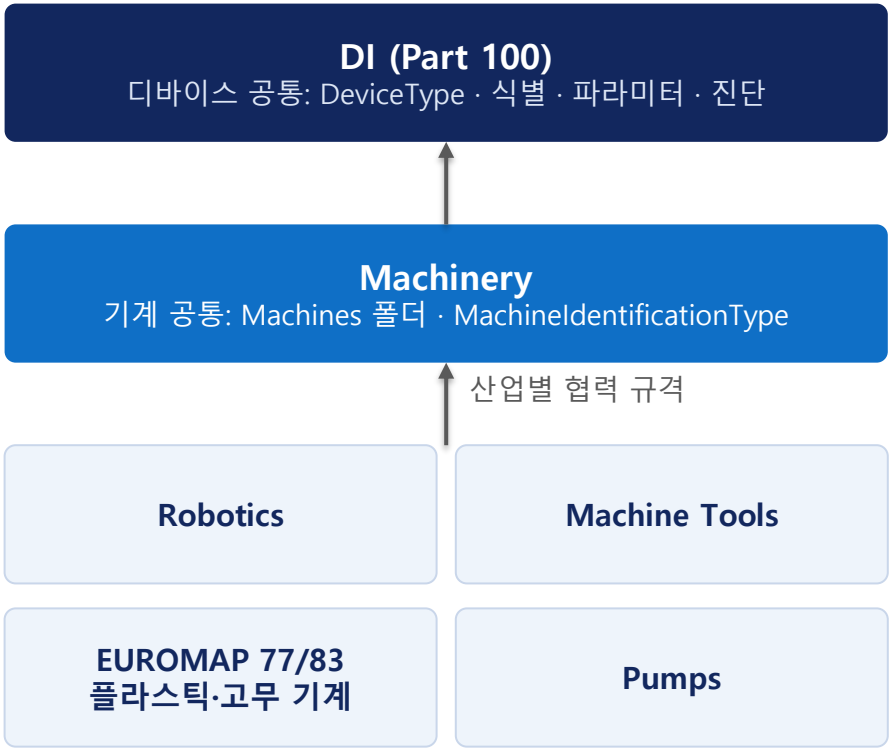
모든 전이 → **ProgramTransitionEventType** (TransitionEventType 하위) · **AuditProgramTransitionEventType** (감사)
실선 = 제어 메서드 · 점선 = 내부 원인 · 메서드는 모두 선택이고 인자를 가질 수 있다

원문: OPC 10000-10 Figure 4 – Program Type · §4.2.3 Figure 3 · Table 1 · §5.2.3.3

정보 모델링

■ 도메인 특화 정보 모델

➢ 같은 종류의 설비는 벤더가 달라도 같은 모델로 표현한다



MachineIdentificationType	의미
Manufacturer	제조사
Model	모델명
SerialNumber	일련번호
YearOfConstruction	제작 연도
ProductInstanceUri	기기 고유 URI
Location	설치 위치

벤더가 달라도 같은 경로
Machines/<설비>/Identification에서 같은 이름의 명판 정보를 읽는다

근거: OPC 10000-100, OPC 40001-1 (Machinery)

■ 도메인 특화 정보 모델

➢ 모델은 설계 → NodeSet2 XML → 서버 임포트 → 검증 순으로 쓴다

1. 설계	2. 교환	3. 구현	4. 검증
모델링 도구로 타입·인스턴스 정의 UaModeler · SiOME 등	NodeSet2 XML로 내보내기 표준 모델: UA-Nodeset 저장소	SDK가 XML을 임포트해 주소공간 생성 asyncua import_xml · open62541	구조 확인과 적합성 점검 UaExpert · UACTT Lite

주의: NodeSet 버전과 SDK의 코어 모델 버전을 맞춘다
사전 검증(asyncua 2.0.1): 최신 DI 1.05.0은 임포트 실패(BadParentNodeIdInvalid), asyncua 저장소가 고정한 DI 1.04.0 · Machinery 1.02.0은 성공

INDUSTRIAL AI RESEARCH CENTER

CONTENTS

I OPC UA 개념

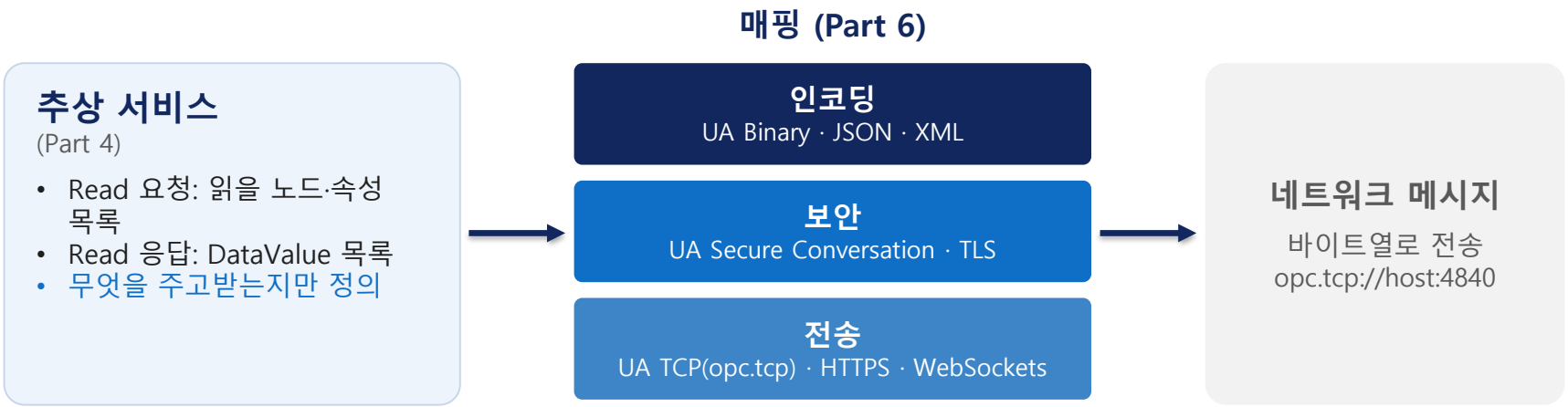
II OPC Foundation

III 정보 모델링

IV 시스템 및 서비스

■ 개념

➢ 서비스는 추상적으로 정의하고, 전송 매핑에 실어 보낸다



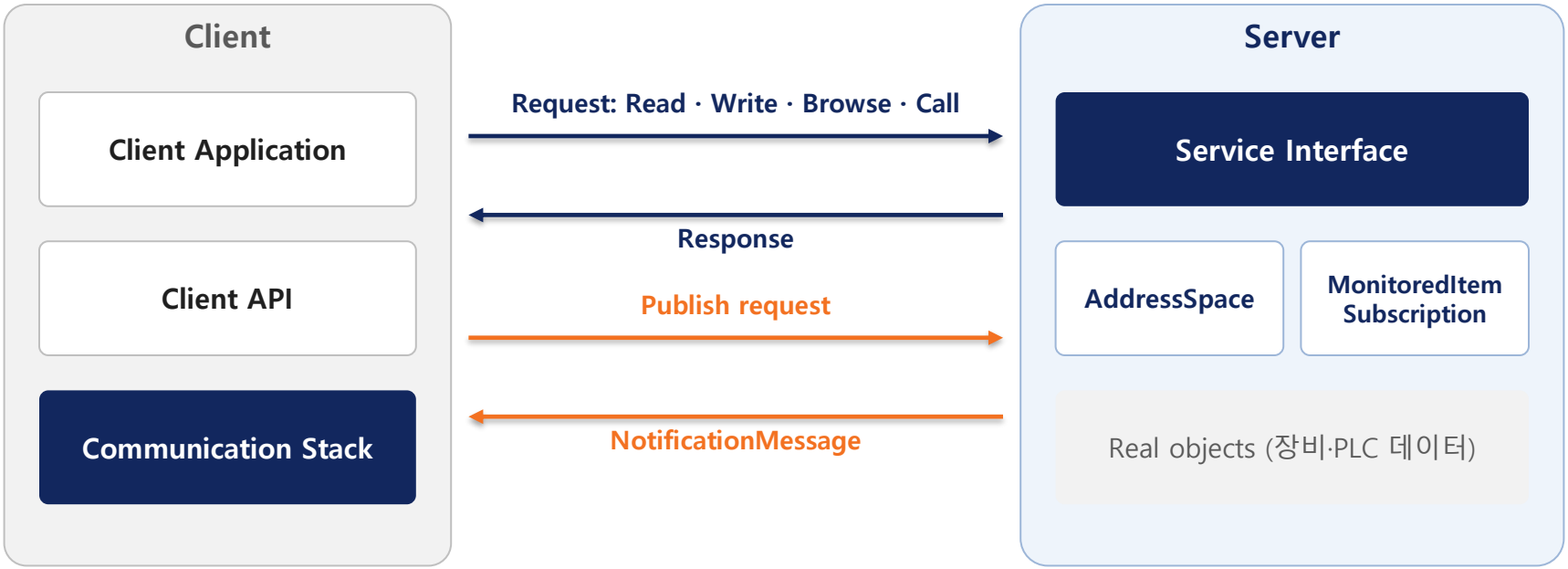
PubSub 매핑 (Part 14)
메시지 UADP · JSON / 전송 UDP · MQTT · AMQP · 이더넷(TSN)

새 전송 기술이 나와도 **서비스와 정보 모델은 그대로**, 매핑만 추가한다

근거: OPC 10000-4 Services · OPC 10000-6 Mappings · OPC 10000-14 PubSub

■ 시스템 . 서버-클라이언트 구조 패턴

➢ ClientServer는 요청·응답과 구독(Subscription)으로 동작한다



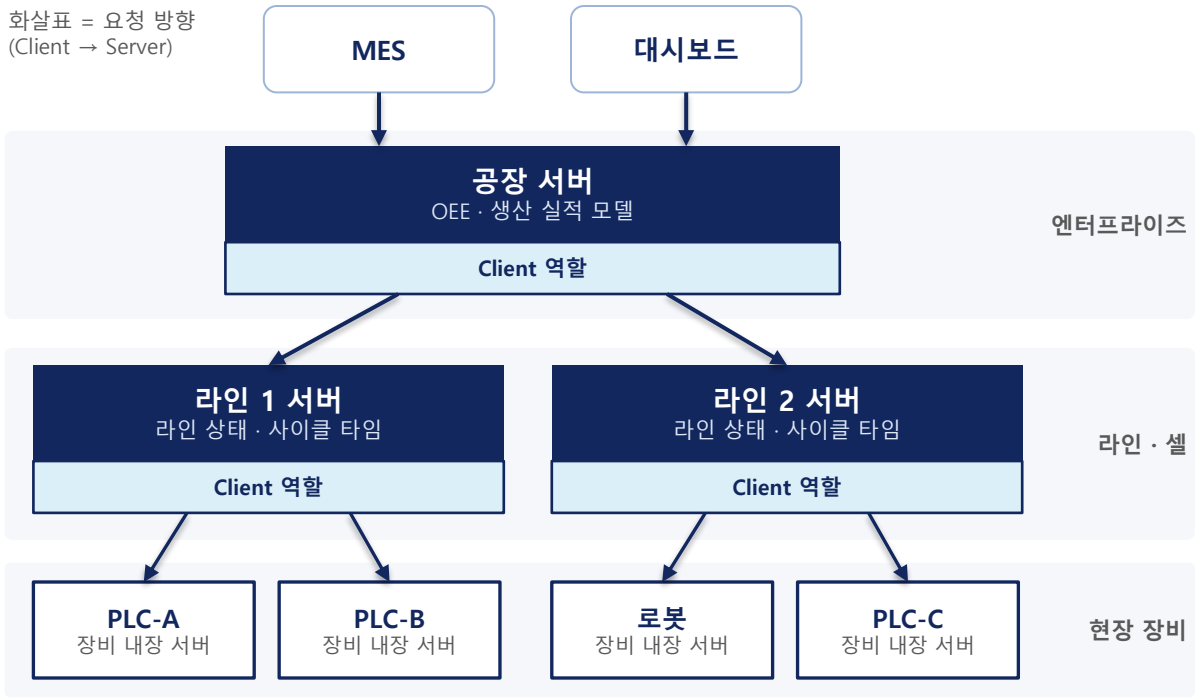
Server to Server (서버 간 연동)
서버가 다른 서버의 Client → 체인 · 서버-서버 · 집합

Redundancy (이중화)
서버 이중화(투명·비투명), 클라이언트 이중화

원문: OPC 10000-1 Figure 2 – OPC UA system architecture · Server to Server: Figure 5·6

■ 시스템 · 체인 서버 구조 패턴

➢ 서버가 아래층 서버의 Client가 되어 층을 이루고, 층마다 상위 데이터를 만든다



위로 Server · 아래로 Client
각 층 서버가 아래층 서버에 접속해 값을 읽고 구독한다

상위 데이터 구성
라인 상태 · OEE처럼 아래에 없던 정보를 새로 만든다

하위 데이터 집계
여러 장비의 값을 모아 한 곳에서 제공한다

서버 간 상호작용 = 한 서버가 다른 서버의 **Client**가 되는 것 → 층으로 이으면 **체인 서버**

원문: OPC 10000-1 Figure 6 – Chained Server example

■ 시스템 . 서버-서버 구조 패턴

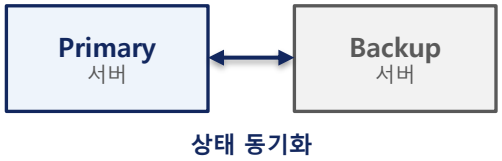
➢ 서버끼리 서로의 Client가 되어 대등하게 정보를 주고받는다



서로 상대의 Client가 된다
(Read · Write · 구독)

ClientServer 모델 안의 연동
세션 · 보안 · 서비스를 그대로 쓴다. 상대 서버에는 일반 클라이언트로 보인다.

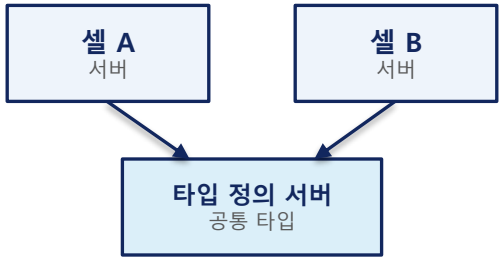
① 이중화



상태 동기화

서버 이중화 쌍이 서로 상태를 맞춘다

② 공통 타입 정의



시스템 공통 타입을 원격 서버가 유지한다

③ 설비 간 연동 (예)



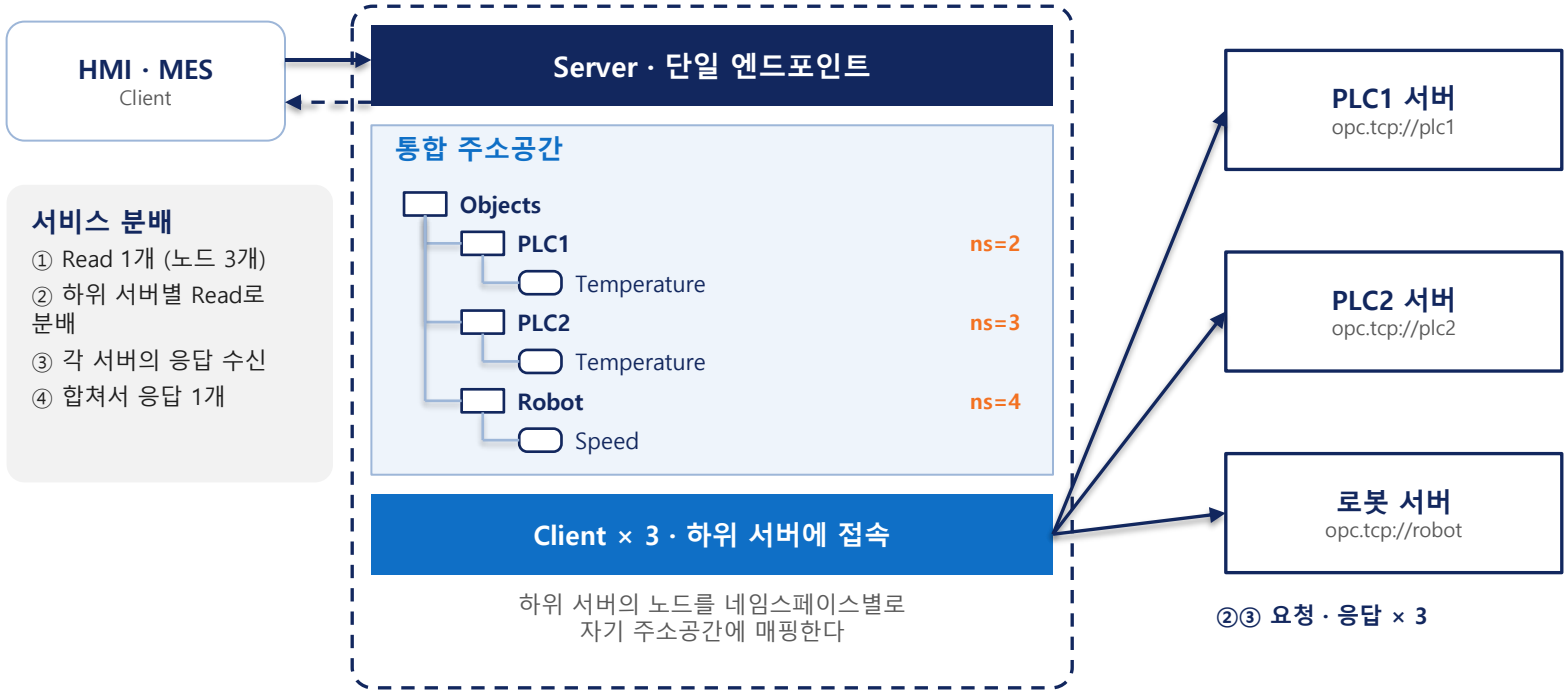
Write: 준비 완료

앞 공정 서버가 다음 공정에 신호를 쓴다

원문: OPC 10000-1 Figure 5 – Peer-to-peer interactions between Servers

■ 시스템 · 집합 서버 구조 패턴

➢ 집합 서버는 여러 하위 서버를 접속점 하나로 묶고, 요청을 나눠 보낸다



단일 접속점
concentrator · Part 1 §5.3.7

서비스 분배
요청 1 → 하위 요청 N · Part 2 §4.14.5

감사 체인
원 Client · 감사 ID 전달 · Part 2 §4.14.3

원문: OPC 10000-2 Figure 8 – Aggregating Servers · Figure 10 – service distribution

INDUSTRIAL AI RESEARCH CENTER

■ 시스템 · Pub-Sub 개념

➢ 발행자와 구독자는 서로를 모른 채 메시지 지향 미들웨어로만 연결된다



구분	ClientServer	PubSub
연결	세션 1:1	세션 없음 · 1:N, N:M
데이터 흐름	요청 · 응답 (구독도 Publish 요청에 응답)	발행자가 주기 · 이벤트로 내보냄
상대 인식	서로 안다 (엔드포인트 · 세션)	서로 모른다 (미들웨어만 안다)
잘 맞는 곳	탐색 · 쓰기 · 메서드 호출	수신자가 많을 때 · 저지연 · 클라우드 연계

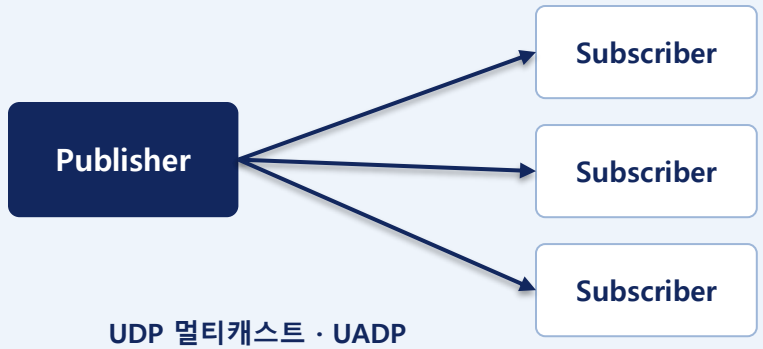
같은 정보 모델 위의 두 방식 · 발행자·구독자 설정은 **ClientServer**로 · 역할은 고정되지 않는다 (§4.4)

원문: OPC 10000-14 Figure 1 – Publish Subscribe model overview · §4.1 · §4.4

■ 시스템 · Pub-Sub 구조 패턴

➢ PubSub에서는 발행자가 DataSet을 여러 구독자에게 1:N으로 보낸다

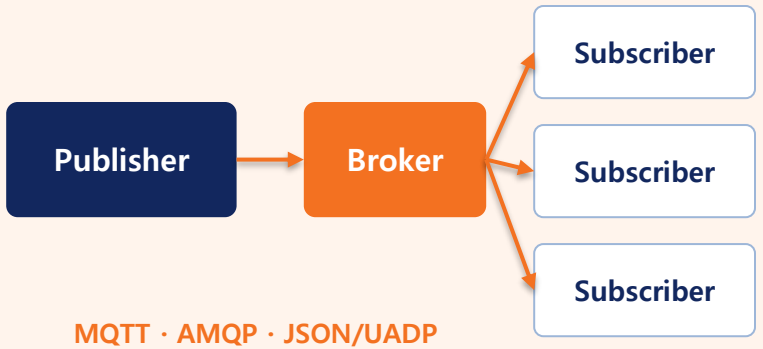
Broker-less (네트워크 인프라)



UDP 멀티캐스트 · UADP

저지연·결정적 → TSN과 결합(OPC UA FX)

Broker-based (Broker 경유)



MQTT · AMQP · JSON/UADP

클라우드·IT 시스템 연계

PublishedDataSet
보낼 데이터 정의 → **DataSetWriter**
DataSetMessage 생성 → **WriterGroup**
NetworkMessage로 묶음 → **NetworkMessage**
UADP · JSON → **DataSetReader**
→ SubscribedDataSet

Publisher 쪽 구성 → NetworkMessage → Subscriber 쪽 구성

보안: **SKS**(Security Key Service)가 그룹 키를 배포해 메시지를 서명·암호화

원문: OPC 10000-14 Figure 14 · Figure 16 (전송 방식) · Figure 3 – DataSet in the process of publishing

■ 시스템 . 시스템 구조

➢ Global Services가 디스커버리·인증서·보안 키를 공통으로 관리한다

Global Services
여러 애플리케이션이 함께 쓰는 공통 인프라 (Part 12)

디스커버리
LDS(호스트) · LDS-ME(서브넷, mDNS) · GDS(전사)로 서버를 찾는다

인증서 관리
GDS가 인증서를 발급·갱신하고 신뢰 목록을 배포한다

KeyCredential 관리
외부 시스템 접속용 자격 증명을 배포한다

권한 서비스
접근 토큰을 발급해 사용자 권한을 위임한다

디바이스 온보딩
신규 기기를 등록하고 운영용 인증서를 부여한다 (Part 21)

별칭 이름
이름으로 노드·서버를 찾는 디렉터리 (Part 17)

SKS
PubSub 그룹 보안 키를 배포한다

근거: OPC 10000-1 §5.7 Global Services · OPC 10000-12

■ 시스템 · 어플리케이션 구조

➢ 목적지향 모듈을 SDK(상단 플랫폼)와 통신 스택(하단 플랫폼) 위에 올려 만든다

목적지향적 소프트웨어 모듈
애플리케이션 고유 기능

HMI 화면 | 데이터 수집 | 장비 드라이버 | 정보 모델

개발자가 작성
목적마다 다르다 · 정보 모델도 여기서 만든다

▲▼ SDK API (Client API · Server API)

상단 플랫폼 = OPC UA SDK
사양 기능을 구현한 재사용 라이브러리

세션 | 구독 | 주소공간 | 보안 · 인증서 | NodeSet 임포트

SDK 벤더가 제공
여러 애플리케이션이 함께 쓴다 · 사양 기능을 구현

▲▼ 스택 API

하단 플랫폼 = 통신 스택 + OS
Part 6 매핑 · 운영체제 기능

인코딩 | SecureChannel | UA TCP | TCP/IP · 스레드 · 암호

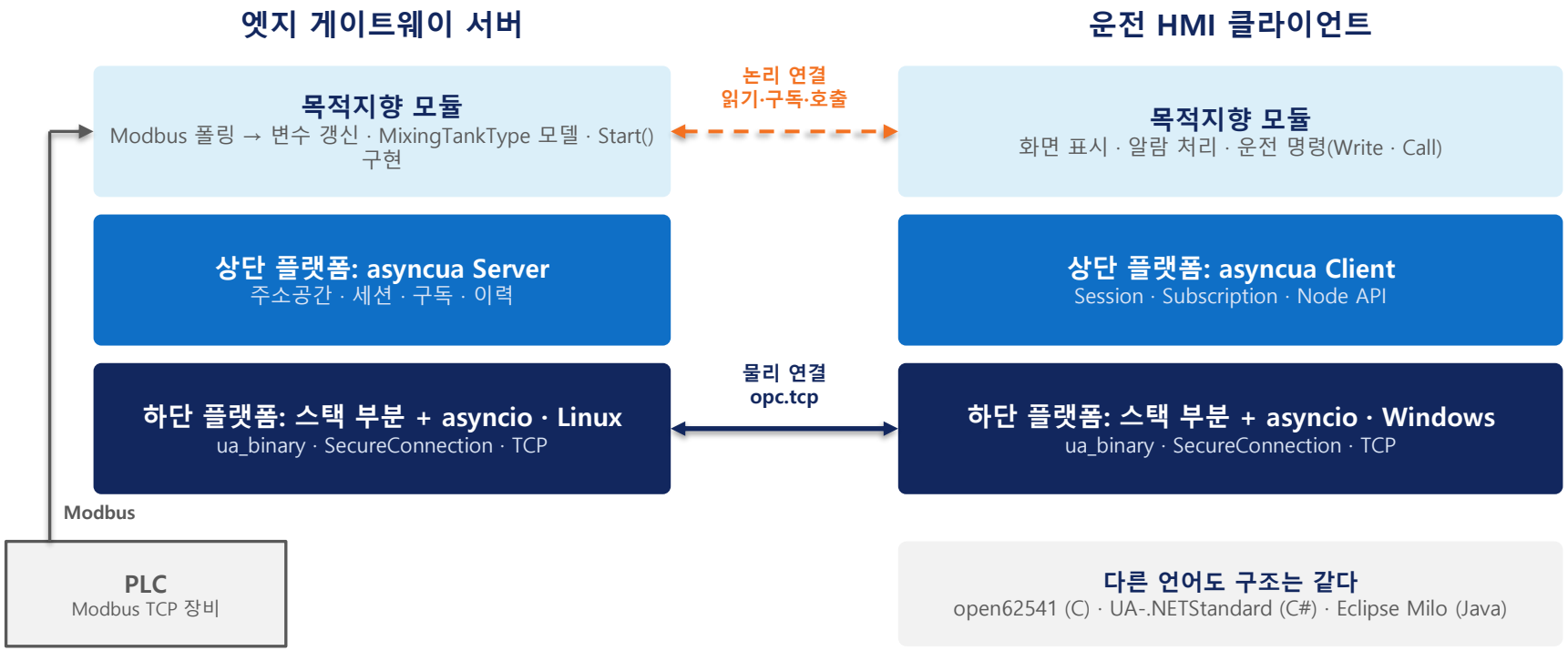
스택 · OS
Part 6 매핑대로 동작 · 거의 손대지 않는다

Part 1 용어: Client / Server Application · Client API / Server API · Communication Stack

근거: OPC 10000-1 §5.2 Client · §5.3.3 Server application · OPC 10000-6 Mappings

■ 시스템 · 어플리케이션 구현 구조 예시

➢ 같은 세 층 구조로 게이트웨이 서버와 HMI 클라이언트를 만든다 (Python 예)



같은 SDK · 스택 위에 **목적 모듈만 바꿔** 서버도, 클라이언트도 만든다

근거: asyncua (opcua-asyncio) · OPC 10000-1 §5.2 · §5.3

■ 시스템 · 스택 구조

➢ 스택은 요청을 인코딩하고 보안을 적용해, 메시지 조각으로 나눠 보낸다

SDK: ReadRequest 객체
NodesToRead = [Temperature.Value]

↓

① 인코딩 · UA Binary
객체 → 바이트열 (Part 6 §5.2)

↓

② 보안 · UA Secure Conversation
조각으로 나누고 서명 · 암호화 (§6.7)

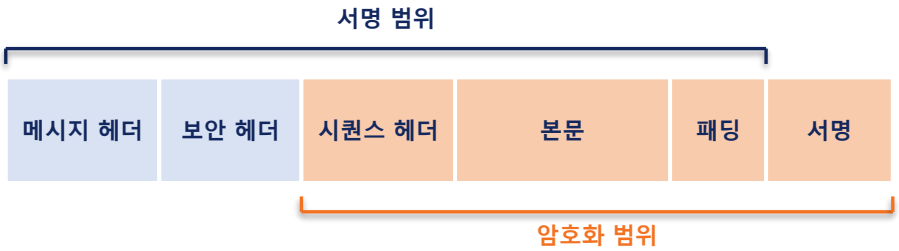
↓

③ 전송 · UA TCP (UACP)
HEL / ACK 후 MSG 조각 전송 (§7.1)

↓

TCP/IP · 포트 4840

MessageChunk 구조 (SignAndEncrypt)



- 메시지 헤더 (12바이트): MSG · OPN · CLO / IsFinal C · F · A / 크기 / SecureChannelId
- 보안 헤더: 대칭 키 TokenId (채널 열 때는 정책 URI · 인증서)
- 시퀀스 헤더 (8바이트): SequenceNumber · RequestId
- 본문: UA Binary로 인코딩한 ReadRequest
- 패딩 · 서명: 암호 블록 크기 맞춤 · 위변조 확인

SecurityMode: **None** · **Sign** (서명) · **SignAndEncrypt** (서명 + 암호화)

큰 메시지는 여러 조각(**C** … **F**)으로 나뉘고, 받는 쪽 스택이 다시 합쳐 디코딩한다

원문: OPC 10000-6 Figure 11 – MessageChunk · §5.2 UA Binary · §7.1.2 UACP

INDUSTRIAL AI RESEARCH CENTER

■ 시스템 · SDK 구조

➢ SDK는 스택 위에 세션·주소공간·구독 관리를 얹어 API로 제공한다

서버 쪽 · asyncua.server — 상단 플랫폼

Server (server.py)
고수준 API: 노드 추가 · NodeSet 임포트 · 시작

InternalServer (internal_server.py) · 서비스 처리

- **AddressSpace** — Attribute · View · Method 서비스
- **SubscriptionService** — MonitoredItemService
- **HistoryManager** — 이력 저장 · 조회
- **InternalSession** — 세션 상태

BinaryServer → UaProcessor
asyncio TCP 수신 · 요청 디코딩 · 서비스 호출

binary_server_asyncio.py · uaprocessor.py

클라이언트 쪽 · asyncua.client — 상단 플랫폼

Client (client.py)
고수준 API: connect · get_node · create_subscription

Node · Subscription (common/)
노드 읽기 · 쓰기 · 탐색, 구독 객체

UaClient (ua_client.py)
서비스 호출: read · write · browse · call · create_subscription

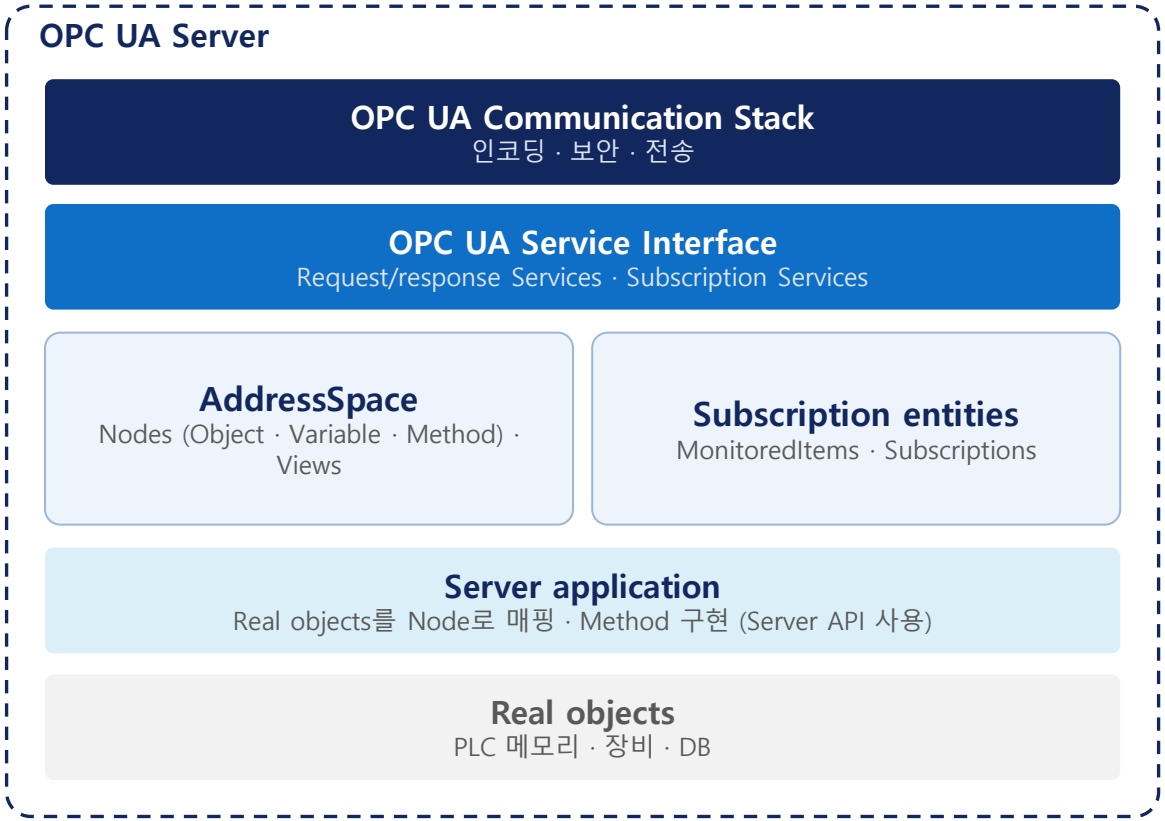
UASocketProtocol
RequestId로 요청 · 응답 짝 맞추기 · asyncio TCP

하단 플랫폼 (스택 부분, 공통)
SecureConnection · MessageChunk (common/connection.py) · ua_binary · uatypes (ua/) · security_policies (crypto/)

근거: asyncua 2.0.1 패키지 구조 (opcua-asyncio)

■ 시스템 . 서버 어플리케이션 구조

➢ 서버는 실제 객체를 주소공간으로 노출하고, 서비스로 접근하게 한다



MonitoredItem
샘플링 · 필터(데드밴드) · 큐잉

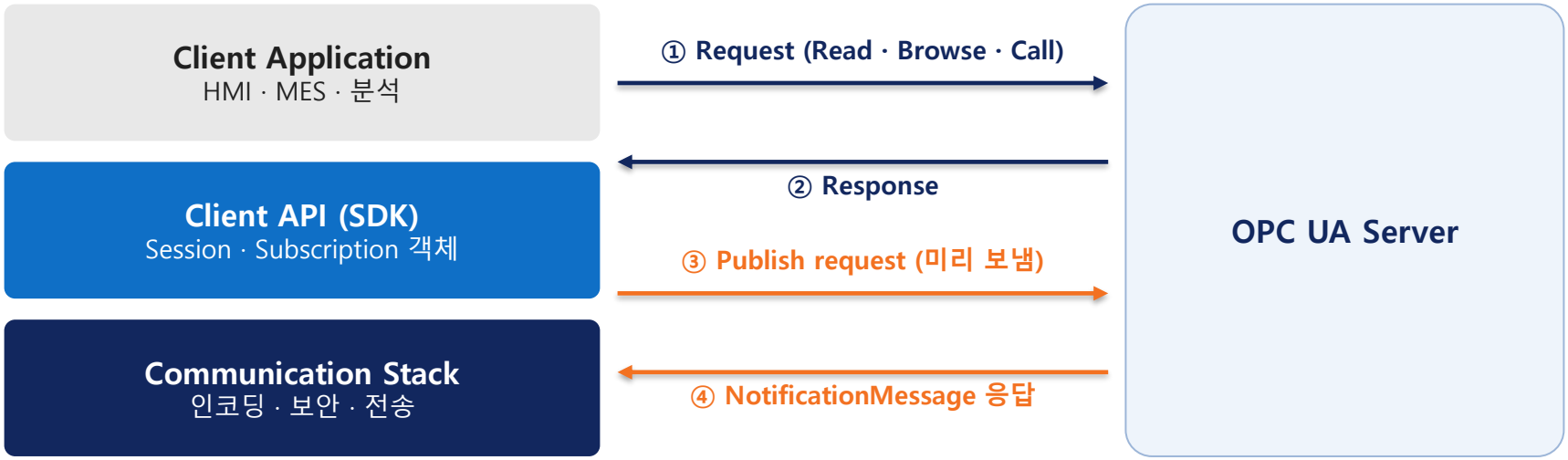
Subscription
모은 알림을 PublishingInterval마다 전달

Server to Server
서버가 다른 서버의 Client가 될 수 있다

원문: OPC 10000-1 Figure 4 – OPC UA Server architecture

■ 시스템 · 클라이언트 어플리케이션 구조

➢ 클라이언트는 API로 서비스를 호출하고, Publish로 알림을 받는다



대표 SDK

asyncua	open62541	UA-.NETStandard	node-opcua	Eclipse Milo
Python	C	C#	JavaScript	Java

원문: OPC 10000-1 Figure 3 – OPC UA Client architecture

■ 서비스 . 개념

➢ 서비스는 요청·응답 메시지 쌍이며, 공통 헤더와 결과 코드를 갖는다

요청 메시지

RequestHeader
authenticationToken · timestamp · requestHandle · returnDiagnostics · auditEntryId · timeoutHint

본문: 서비스별 파라미터 (예: nodesToRead)

응답 메시지

ResponseHeader
timestamp · requestHandle · serviceResult(StatusCode) · serviceDiagnostics · stringTable

본문: results[] — 항목마다 StatusCode

StatusCode (32비트)

심각도 2비트	SubCode · 원인	Info 비트

Good (00) | Uncertain (01) | Bad (10)

serviceResult와 results[i]는 따로 확인

근거: OPC 10000-4 §5.2 요청·응답 헤더 · §7.38 StatusCode

■ 서비스 . 서비스 셋

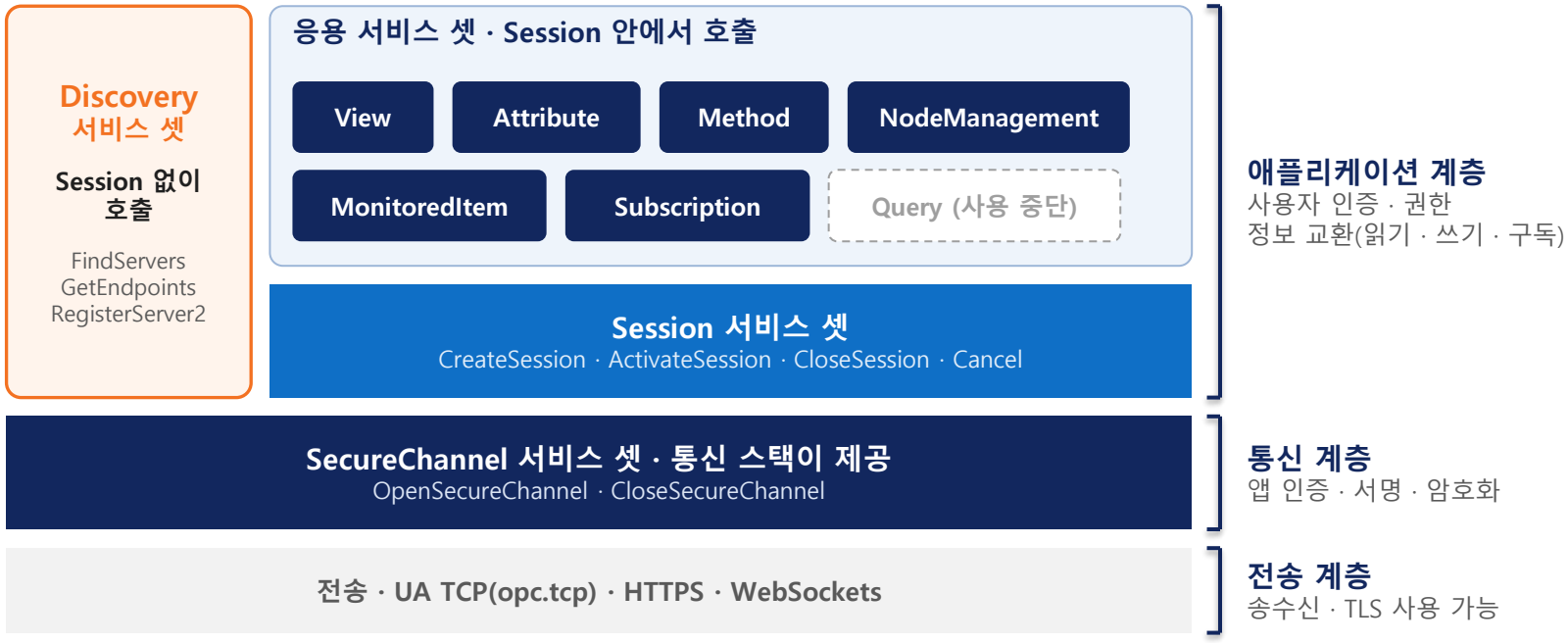
➢ ClientServer 서비스는 10개 서비스 셋으로 묶인다

서비스 셋	역할	대표 서비스
Discovery	서버·엔드포인트 찾기	FindServers · FindServersOnNetwork · GetEndpoints · RegisterServer2
SecureChannel	보안 채널 수립	OpenSecureChannel · CloseSecureChannel
Session	세션 수립·인증	CreateSession · ActivateSession · CloseSession · Cancel
NodeManagement	노드·참조 추가·삭제	AddNodes · AddReferences · DeleteNodes · DeleteReferences
View	주소공간 탐색	Browse · BrowseNext · TranslateBrowsePathsToNodeIds · RegisterNodes
Query	조건 기반 조회	QueryFirst · QueryNext (1.05.06에서 deprecated)
Attribute	속성 읽기·쓰기, 이력	Read · Write · HistoryRead · HistoryUpdate
Method	메서드 호출	Call
MonitoredItem	감시 항목 관리	Create/Modify/DeleteMonitoredItems · SetMonitoringMode · SetTriggering
Subscription	알림 전달 관리	Create/ModifySubscription · SetPublishingMode · Publish · Republish · TransferSubscriptions

원문: OPC 10000-4 Figure 1~8 – Service Set model · 근거: §5.5~5.14

■ 서비스 · OPC UA 서비스 계층

➢ 서비스 셋은 SecureChannel → Session → 응용 서비스 순으로 쌓인다



응용 서비스는 **Session** 안에서, Session은 **SecureChannel** 위에서 — Discovery만 Session 없이 호출

원문: OPC 10000-4 Figure 1~8 – Service Set model · 근거: OPC 10000-2 §4.5.2 Client/Server

■ 서비스 · Discovery 서비스 셋 개념

➢ Discovery는 Session 없이 서버와 엔드포인트 정보를 얻는 서비스 셋이다



EndpointDescription — 엔드포인트 한 개의 구성

어디로 접속

endpointUrl
접속 URL

transportProfileUri
전송 프로파일

server
서버 애플리케이션 정보

SecureChannel 설정

securityPolicyUri
보안 정책(알고리즘 묶음)

securityMode
None · Sign · SignAndEncrypt

serverCertificate
서버 인증서

Session 설정

userIdentityTokens
허용하는 사용자 토큰

→ **ActivateSession**
이 중 하나를 골라 보낸다

고르는 기준

securityLevel
같은 서버 안 상대값

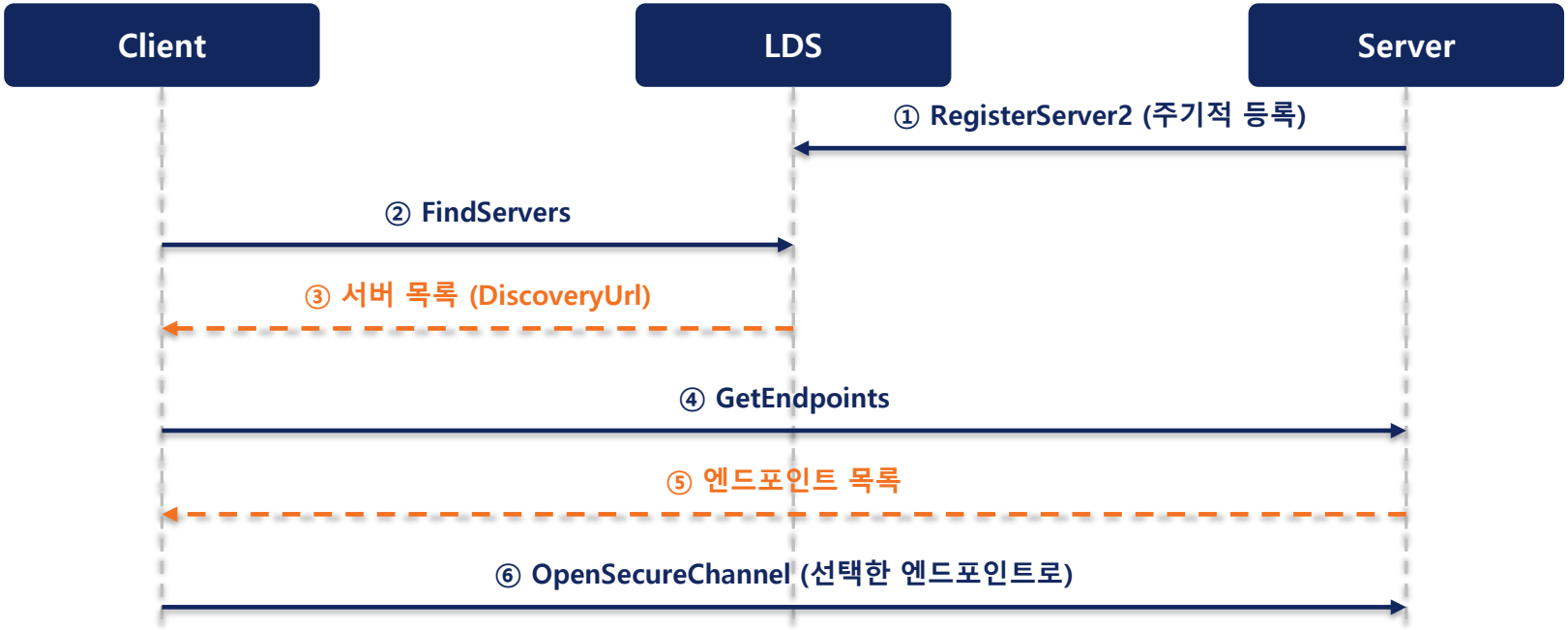
값이 클수록 안전
0 = 권장하지 않음

검증: 클라이언트는 CreateSession 응답의 serverEndpoints와 이 목록을 비교한다

원문: OPC 10000-4 Figure 1 – Discovery Service Set · 근거: §5.5 Discovery · §7.14 EndpointDescription

■ 서비스 · Discovery 시퀀스 다이어그램

➢ 클라이언트는 Discovery로 서버와 엔드포인트를 찾는다



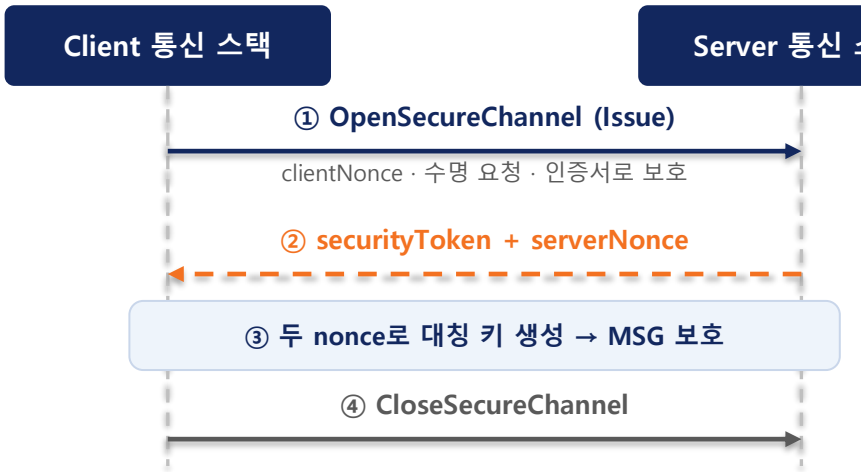
서버 주소를 이미 알면
LDS 없이 ④ GetEndpoints부터 시작한다 (실습 방식)

범위별 검색
서브넷: LDS-ME(mDNS) · 전사: GDS

원문: OPC 10000-4 Figure 9 – Discovery process · OPC 10000-12 Figure 3 – Local Discovery Process

■ 서비스 · SecureChannel 서비스 셋 개념

➢ SecureChannel은 통신 스택이 열고, 보안 토큰을 주기적으로 갱신한다

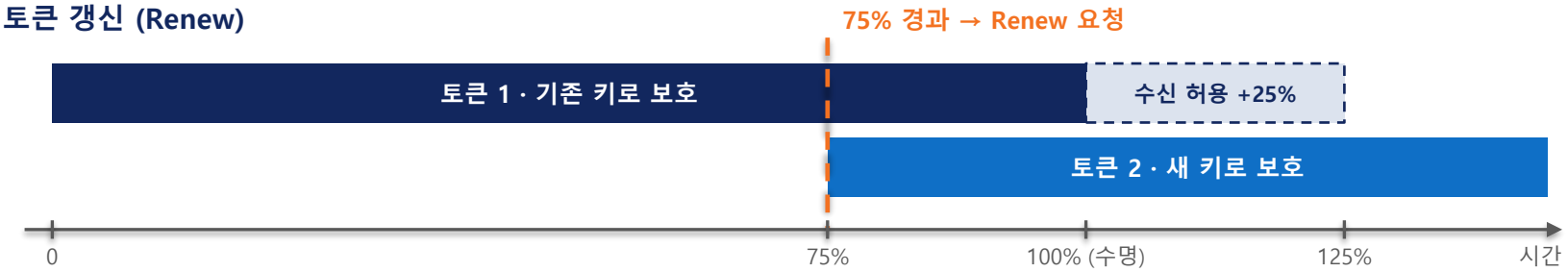


securityToken (ChannelSecurityToken)

필드	의미
channelId	SecureChannel 식별자
tokenId	현재 키 묶음 식별자
createdAt	발급 시각
revisedLifetime	유효 시간 (ms)

requestType
Issue 새 채널 · **Renew** 토큰 갱신

보안 토큰 갱신 (Renew)



원문: OPC 10000-4 Figure 2 – SecureChannel Service Set · 근거: §5.6.2 OpenSecureChannel

■ 서비스 · SecureChannel과 Session 관계

➢ Session은 SecureChannel 위에서 만들지만, 채널이 바뀌어도 이어 쓸 수 있다

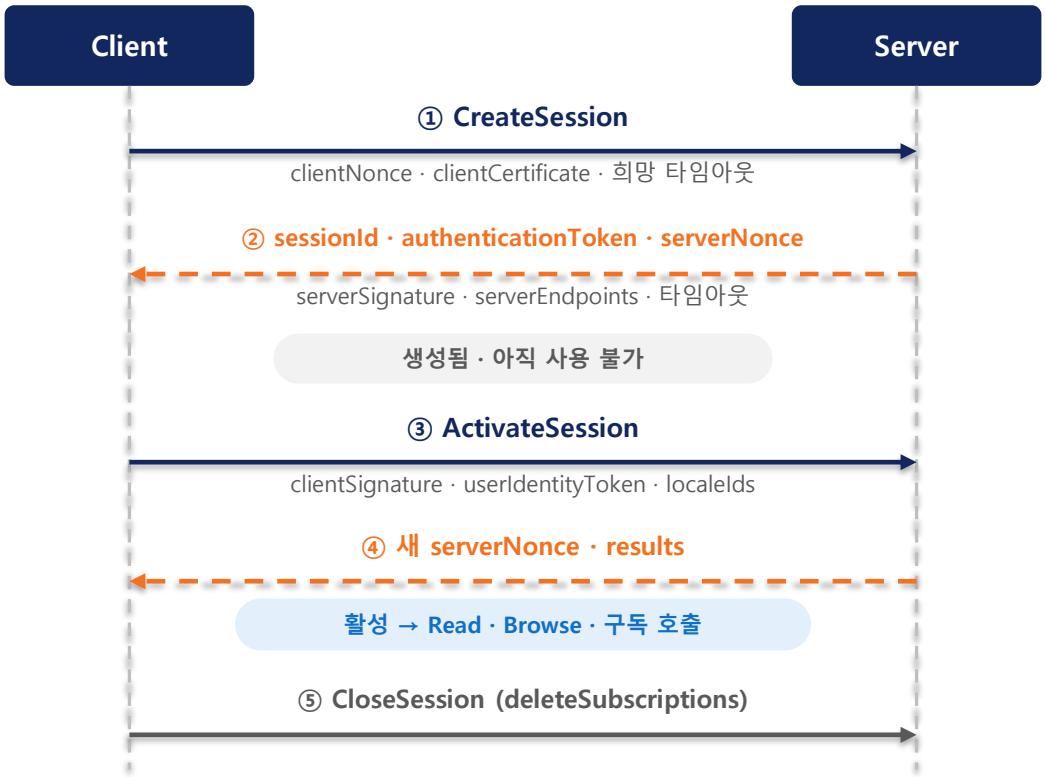


구분	SecureChannel	Session
계층	통신 계층 · 통신 스택이 제공	애플리케이션 계층 · 애플리케이션이 처리
인증	애플리케이션 인증 (인증서)	사용자 인증 (사용자 ID 토큰)
보호 · 판단	서명 · 암호화 (무결성 · 기밀성)	권한 판단 (누가 무엇을)
식별	channelId · tokenId	sessionId · authenticationToken
수명	토큰 수명 75%에서 Renew	요청이 없으면 세션 타임아웃
서비스	Open · CloseSecureChannel	Create · Activate · CloseSession · Cancel

원문: OPC 10000-4 Figure 13 – SecureChannel and Session Services · §5.7.3 · Part 2 §4.5.2

■ 서비스 · Session 서비스 셋 개념

➢ CreateSession으로 만들고, ActivateSession으로 사용자를 붙여야 쓸 수 있다



Session 서비스 셋

CreateSession
Session 생성 · 서로의 인증서 확인

ActivateSession
사용자 지정 · 채널 변경 · 활성화

CloseSession
종료 · 구독 삭제 여부 선택

Cancel
처리 중인 요청 취소

사용자 ID 토큰 (ActivateSession)

Anonymous 익명	**UserName** ID · 비밀번호
X509 사용자 인증서	**Issued** 외부 발급 토큰

원문: OPC 10000-4 Figure 3 – Session Service Set · 근거: §5.7.2 CreateSession · §5.7.3 ActivateSession

INDUSTRIAL AI RESEARCH CENTER

■ 서비스 · NodeManagement 서비스 셋 개념

➢ NodeManagement는 클라이언트가 주소공간에 노드와 참조를 더하고 지운다

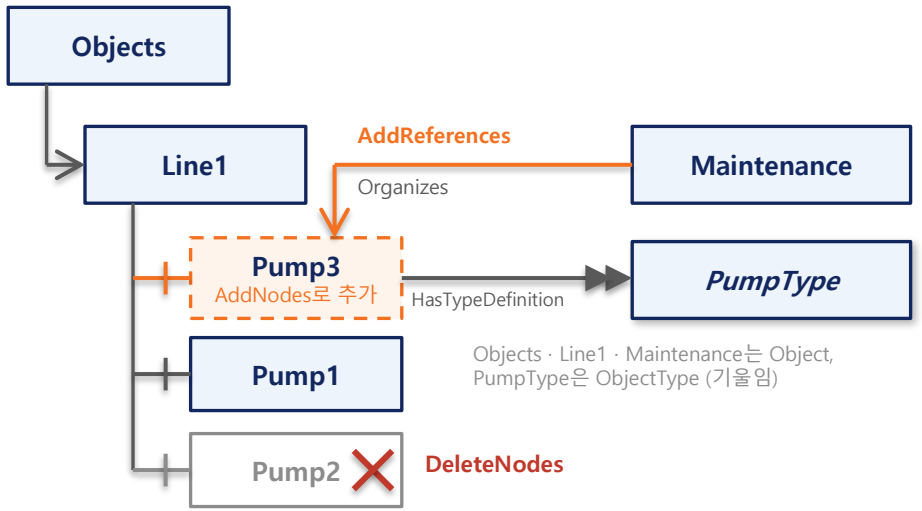
AddNodesItem — 요청 항목 하나

필드	그림의 예
parentNodeId	Line1
referenceTypeId	HasComponent
requestedNewNodeId	null → 서버 할당
browseName	Pump3
nodeClass	Object
nodeAttributes	DisplayName 등
typeDefinition	PumpType

결과: statusCode · addedNodeId

AddNodes

서버 주소공간



AddNodes 노드 추가 | **AddReferences** 참조 추가 | **DeleteNodes** 노드 삭제 | **DeleteReferences** 참조 삭제

추가한 노드는 만든 클라이언트가 끊겨도 남는다 · 실제 변경 범위는 서버마다 다르다 · 운영 서버는 흔히 막아 둔다

원문: OPC 10000-4 Figure 4 – NodeManagement Service Set · 근거: §5.8 NodeManagement

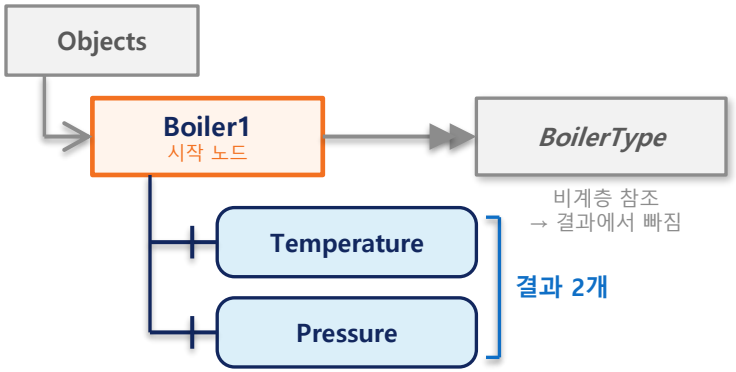
INDUSTRIAL AI RESEARCH CENTER

■ 서비스 · View 서비스 셋 개념

➢ View 서비스로 참조를 따라 주소공간을 탐색하고, 경로를 NodeId로 바꾼다

① Browse · BrowseNext — 참조 따라가기

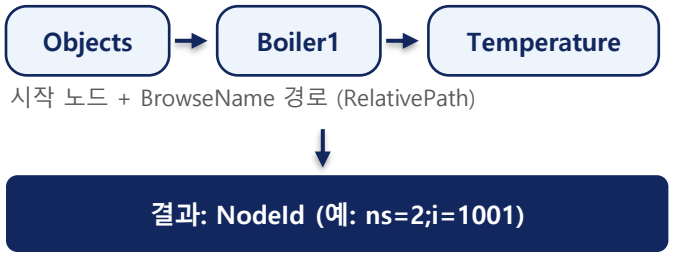
nodeId: Boiler1 · browseDirection: Forward
referenceTypeId: HierarchicalReferences
includeSubtypes: true · view: 전체 주소공간



응답: ReferenceDescription[] (NodeId · BrowseName ...)

많으면 continuationPoint → BrowseNext로 이어 받기

② TranslateBrowsePathsToNodeIds — 경로 → NodeId



③ RegisterNodes · UnregisterNodes — 반복 접근 최적화



이후 Read · Write · Call에 별칭 사용

다 쓰면 UnregisterNodes로 해제

원문: OPC 10000-4 Figure 5 – View Service Set · 근거: §5.9 View

INDUSTRIAL AI RESEARCH CENTER

■ 서비스 · Query 서비스 셋 개념

➢ Query는 타입 기반 조회를 정의했지만, 1.05.06에서 사용 중단(deprecated)됐다

Query 서비스 셋 (Part 4 Annex B)

- **QueryFirst**: View · 대상 타입 · 조건(ContentFilter) · 최대 결과 수
- 결과 + ContinuationPoint 반환
- **QueryNext**: 나머지 결과를 이어 받기

QueryFirst 요청 예

NodeTypes : MixingTankType
Filter : Temperature > 80
MaxDataSetsToReturn : 100

→ **QueryDataSets + ContinuationPoint**

실무 대안 — Query 없이 같은 결과 얻기

Browse · BrowseNext
구조 탐색

→

TranslateBrowsePathsToNodeIds
경로 → NodeId

→

Read
읽은 뒤 클라이언트에서 필터

HistoryRead
과거 데이터 조회

1.05.06에서 **사용 중단(deprecated)** — 현장에 채택되지 않음 → **Browse·Read 기반으로 설계**

근거: OPC 10000-4 §5.10 Query Service Set

INDUSTRIAL AI RESEARCH CENTER

■ 서비스 · Attribute 서비스 셋 개념

➢ Attribute 서비스로 노드의 속성을 읽고 쓰며, 과거 값(이력)도 다룬다

Read 요청
ReadValueId

nodeId	attributeId	indexRange	dataEncoding
어느 노드	어느 속성	배열 일부	인코딩

Read 응답
DataValue

value	statusCode	sourceTimestamp	serverTimestamp
값	품질 상태	소스 시각	서버 시각

Write 항목
WriteValue
→ 항목별 StatusCode

nodeId	attributeId	indexRange	value
어느 노드	어느 속성	배열 일부	쓸 값 (DataValue)

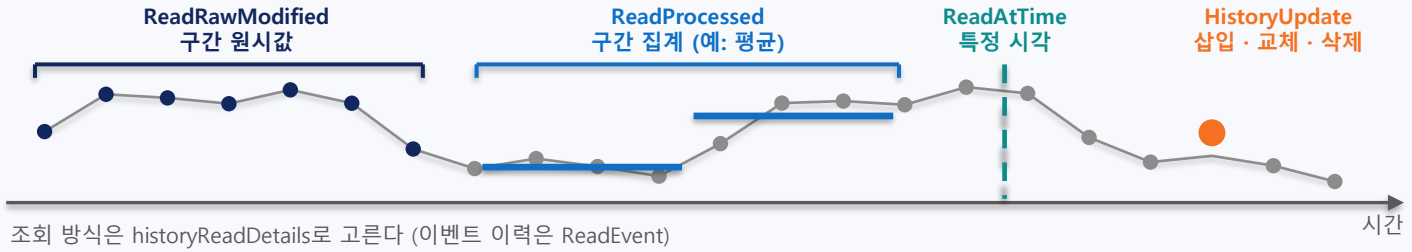
+ 요청 공통 파라미터
maxAge · timestampsToReturn

maxAge — 몇 ms 전 값까지 허용?

- **maxAge = 0** → 데이터 소스에서 새로 읽기
- **maxAge 큼** → 서버 캐시 값 사용

그 사이 값: 그보다 오래된 캐시는 안 씀

HistoryRead
HistoryUpdate
과거 값 다루기



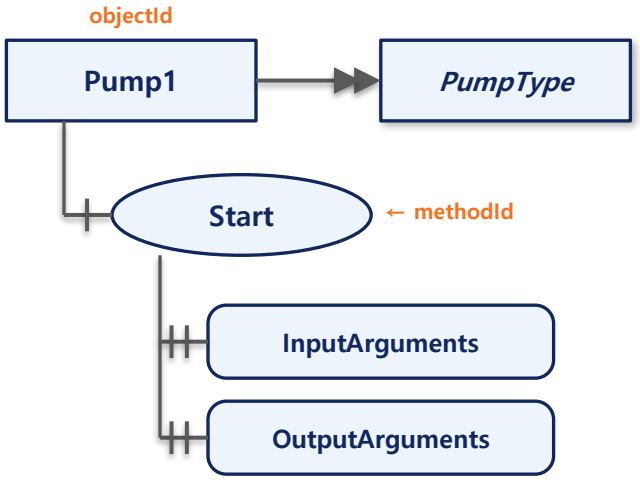
조회 방식은 historyReadDetails로 고른다 (이벤트 이력은 ReadEvent)

원문: OPC 10000-4 Figure 6 – Attribute Service Set · 근거: §5.11.2 Read · §5.11.3 HistoryRead

■ 서비스 · Method 서비스 셋 개념

➢ Call 하나로 객체의 메서드를 호출하고, 실행이 끝나면 결과를 받는다

주소공간의 메서드



HasComponent: 객체가 메서드를 가짐
HasProperty: 인자 모양(이름 · 타입)

Call 요청 · CallMethodRequest
objectId Pump1 — 메서드를 가진 Object(또는 ObjectType)
methodId Start
inputArguments [1200] — InputArguments 순서대로

↓

서버가 실행 → 끝난 뒤 응답 (성공 · 실패)

↓

Call 응답 · CallMethodResult
statusCode 실행 결과
inputArgumentResults 인자 오류(Bad_InvalidArgument) 때 인자별 결과
outputArguments 출력 값 — statusCode가 Bad면 비어 있음

동기 호출
끝날 때까지 기다린다 → 긴 작업은 Program(Ⅲ장)

여러 호출을 한 번에
methodsToCall[]에 묶어 한 요청으로 보낸다

원문: OPC 10000-4 Figure 7 – Method Service Set · 근거: §5.12.2 Call

■ 서비스 · MonitoredItem 서비스 셋 개념

➢ MonitoredItem은 무엇을 어떻게 감시할지 정하고, Subscription에 담겨 보고된다



MonitoredItemCreateRequest — 생성 요청 항목

- **itemToMonitor** — 무엇을 (ReadValueId)
- **monitoringMode** — 감시 모드
- **requestedParameters** — 어떻게
  - clientHandle
  - samplingInterval
  - filter
  - queueSize
  - discardOldest

→ 결과: monitoredItemId · 실제 적용값(revised...) · filterResult

MonitoredItem 서비스 셋

CreateMonitoredItems · 만들기
ModifyMonitoredItems · 바꾸기
SetMonitoringMode · 모드 전환
SetTriggering · 트리거 연결
DeleteMonitoredItems · 지우기

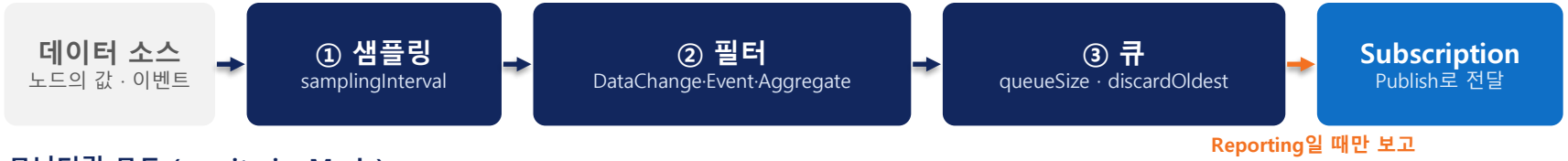
모두 subscriptionId로 대상 Subscription을 지정

원문: OPC 10000-4 Figure 8 – MonitoredItem and Subscription Service Sets · 근거: §5.13.2

INDUSTRIAL AI RESEARCH CENTER

■ 서비스 · MonitoredItem 메커니즘

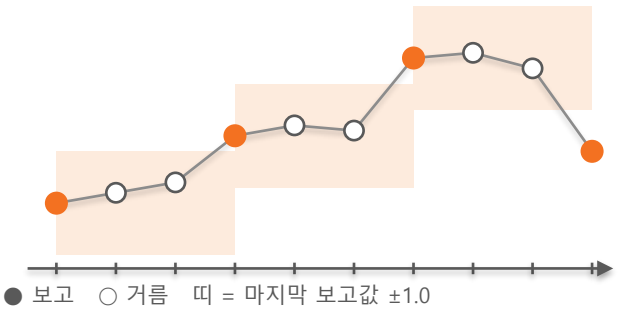
➢ 샘플링 → 필터 → 큐를 거친 알림만 Subscription이 보낸다



모니터링 모드 (monitoringMode)

Disabled	Sampling	Reporting
샘플링 안 함	샘플링 · 큐잉만, 보고 안 함	샘플링 · 큐잉 · 보고

샘플링 · 데드밴드 (Absolute 1.0)



큐 넘침 · queueSize 3, 새 값 v4

discardOldest = TRUE



discardOldest = FALSE



overflow 표시 = StatusCode InfoBits

트리거링 (SetTriggering)



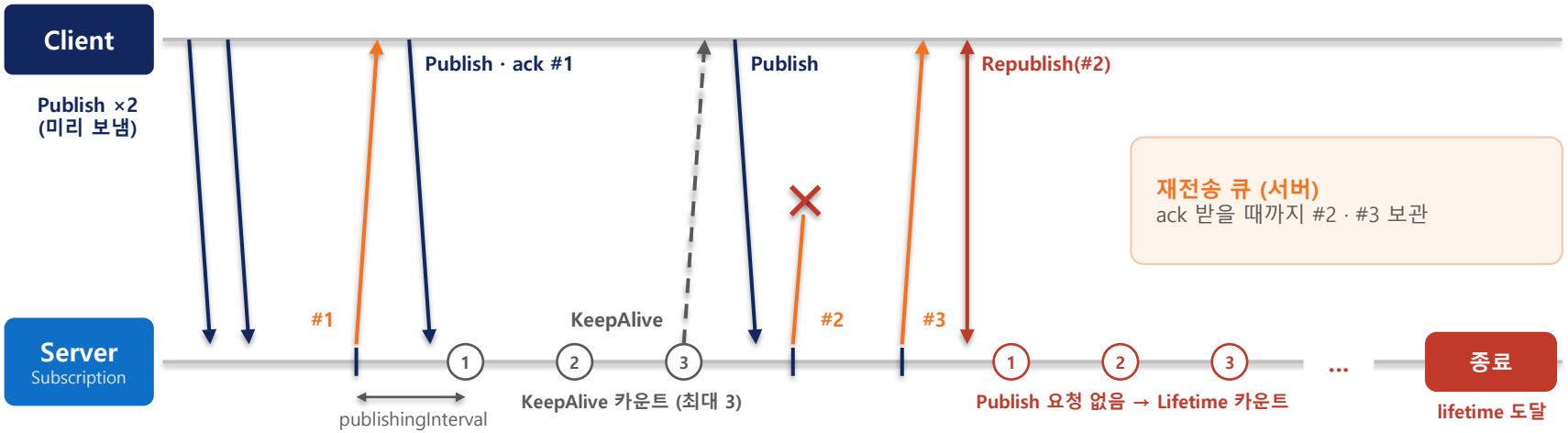
A가 보고될 때
B의 큐도 함께 보고

원문: OPC 10000-4 Figure 15 – MonitoredItem model · Figure 17 · Figure 18

INDUSTRIAL AI RESEARCH CENTER

■ 서비스 · Subscription 메커니즘

➢ 구독은 클라이언트의 Publish 요청과 서버의 응답으로 돌아간다



① Publish 요청 큐
클라이언트가 요청을 미리 보내 두면, 주기마다 하나씩 꺼내 응답한다

② maxKeepAliveCount
이 주기 수 동안 알림이 없으면 KeepAlive(다음 순번 포함)를 보낸다

③ ack · Republish
다음 Publish로 받은 순번을 확인한다. 빠진 순번은 Republish로 다시 받는다

④ lifetimeCount
이 주기 수 동안 Publish 요청이 없으면 종료한다. KeepAlive의 3배 이상

근거: OPC 10000-4 §5.14.1 Subscription model · §5.14.2 CreateSubscription

■ 서비스 운영

➢ 연결은 단계적으로 수립되고, 단계마다 실패 원인이 다르다



대표 실패 (검증값)

일치하는 엔드포인트 없음
BadSecurityChecksFailed (서버에 따라)
BadCertificateUntrusted BadCertificateUriInvalid
BadUserAccessDenied BadIdentityTokenRejected

운영 중 관리

- 채널 토큰은 만료 전 갱신
- 요청이 없으면 세션 타임아웃

원문: OPC 10000-1 Figure 8 – SecureChannel and Session Services

■ 서비스 운영

➢ 결과 코드·진단 정보·감사 이벤트로 문제를 추적한다

StatusCode	의미	먼저 볼 곳
BadNodeIdUnknown	없는 노드	NodeId · 네임스페이스 인덱스
BadTypeMismatch	데이터형 불일치	쓰기 값의 형식
BadNotWritable	쓰기 불가 노드	AccessLevel
BadUserAccessDenied	사용자 권한 없음	UserAccessLevel · 계정
BadTimeout	응답 시간 초과	네트워크 · timeoutHint
BadSessionIdInvalid	세션 무효·만료	재연결 · 세션 타임아웃
BadTooManyOperations	요청 항목 수 초과	ServerCapabilities 한도
BadCertificateUntrusted	신뢰되지 않은 인증서	신뢰 목록(Trust List)

Good

Uncertain

Bad

상위 2비트
심각도

진단: **returnDiagnostics** · **ServerDiagnostics** 노드 · 보안 행위는 **감사(Audit) 이벤트**로 기록

근거: OPC 10000-4 §7.38 StatusCode

■ 핵심 요약

➢ OPC UA는 데이터를 의미와 함께 안전하게 전달하는 산업 데이터 연계의 공통 언어다

Ⅰ OPC UA 개념

- 통신 + 정보 모델 + 보안
- 현장~클라우드 수직적 통합
- 객체 = 변수 · 메서드 · 이벤트

Ⅱ OPC Foundation

- 규격 개발 · 인증 · 표준화 협력
- 협력 규격은 산업 단체와 공동 개발
- 신규 개발은 1.05 기준

Ⅲ 정보 모델링

- 노드·참조 그래프(주소공간)
- 기본 → 접근 유형 → 협력 규격 → 사용자 모델
- 상태 기계 · DA · A&C · 프로그램 표준 모델

Ⅳ 시스템 및 서비스

- 구조 패턴(체인 · 서버-서버 · 집합) + PubSub
- 앱 = 목적 모듈 + SDK + 스택
- 서비스: 채널 → 세션 → 응용 서비스

참고 자료

■ 사양과 도구

➢ 사양은 온라인 레퍼런스에서 파트별로 찾아 읽는다

구분	자료	주소
사양	OPC UA 온라인 레퍼런스 (OPC 10000 시리즈)	reference.opcfoundation.org
사양	Part 1 개요와 개념 (1.05.06) · Part 4 서비스 (1.05.07)	reference.opcfoundation.org/Core/Part4/v105
그림	슬라이드 하단 '원문' 링크 → 해당 절의 원문 그림	그림 번호: 파트별 최신판 (1.05.00~1.05.07)
모델	UA-Nodeset (코어·협력 규격 NodeSet2 XML)	github.com/OPCFoundation/UA-Nodeset
단체	OPC Foundation (미션 · 조직 · 인증 소식)	opcfoundation.org
도구	UaExpert (범용 클라이언트)	unified-automation.com
도구	Prosys OPC UA Simulation Server	prosysopc.com
SDK	asyncua (Python) · 예제: examples 폴더	github.com/FreeOpcUa/opcua-asyncio
도서	Mahnke·Leitner·Damm, OPC Unified Architecture, Springer, 2009	개념서 (최신 사양과 차이 주의)

감사합니다

Q&A